;; CS 135 · Fall 2026 · Lectures L01–L12

(midterm study-guide)

Every "You should know" point from the end of each lecture, explained in full: the steps for things you have to do, the reasoning for things you have to understand, plus the recurring problem shapes, shortcuts and traps. Tick a topic once you can do it without notes.

STEPS SHORTCUT TRAP ON THE EXAM

;; L01

Values and expressions

The model of computation starts here: numbers, prefix expressions, and rewriting an expression one small step at a time until only a value is left.

YOU SHOULD KNOW

- Exact numbers Nat, Int, Rat and how they relate as subtypes
- Writing maths in prefix notation
- Evaluating expressions with substitution steps
- Defining constants with `define`
- `+ - * / quotient remainder sqr expt`, plus `max` and `min`
- `< > = <= >=` and Bool expressions with `true false and or not`

☐ L01 · 1

Exact numbers: Nat, Int, Rat and subtypes

- **Nat** (natural numbers): 0, 1, 2, 3, … **0 is a Nat** in this course. Defined recursively: 0 is a Nat; if n is a Nat then n + 1 is a Nat. (That definition is exactly why the recursion template in L04 works.)
- **Int**: …, −2, −1, 0, 1, 2, …
- **Rat**: any fraction p/q. Racket keeps these **exact**: `(/ 1 3)` produces `1/3`, never 0.333…
- **Subtypes**: Nat ⊆ Int ⊆ Rat (and from L02, Rat ⊆ Num). Every Nat is an Int, every Int is a Rat. A function whose contract says it consumes a Rat automatically accepts Ints and Nats; one that consumes a Nat does *not* promise to work on −3 or 1/2.
- Exact numbers can be arbitrarily large (limited only by memory): `(expt 2 100)` is computed exactly.
- `3/2` is a *number* (one and a half). `(/ 3 2)` is an *expression* that evaluates to that number. DrRacket may print it as a mixed fraction.

SHORTCUT

In the teaching languages, decimal literals are exact too: `1.5` is exactly 3/2 and `(* 1.5 2)` is `3`. Only results marked `#i` are inexact (see L02).

TRAP

Dividing Nats does not give a Nat. `(/ 7 2)` is `7/2`. So the contract for an average is `-> Num` (or Rat), never `-> Nat`. Classic quiz question: "Is −3 a Nat? Is 4 a Rat? Is 3/2 an Int?" (No, yes, no.)

L01 · 2

Writing maths in prefix notation

Racket puts the operator first and wraps every application in parentheses: infix `(6 / 2) · (1 + 2)` becomes `(* (/ 6 2) (+ 1 2))`. Prefix needs no order-of-operations rules because the parentheses say everything.

STEPS: INFIX TO PREFIX

1. Fully bracket the infix expression using BEDMAS so each pair of brackets holds exactly one operator. Implicit multiplication like `2(1+2)` becomes an explicit ·.
2. Inside each bracket, move the operator to the front: `(a op b)` → `(op a b)`.
3. Rename: · or × → `*`, ÷ → `/`, x^2 → `(sqr x)`, x^y → `(expt x y)`, $\sqrt{x}$ → `(sqrt x)`, $|x|$ → `(abs x)`.
4. Optional: merge chains of the same operator, since `+ *` accept many arguments: `(+ (+ a b) c)` → `(+ a b c)`.

```
;; 3x² - 2x + 7            with x defined
;; ((3 · x²) - (2 · x)) + 7
(+ (- (* 3 (sqr x)) (* 2 x)) 7)

;; 6 ÷ 2(1 + 2)  →  ((6 / 2) · (1 + 2))
(* (/ 6 2) (+ 1 2))        ⇒ 9
```

- `+ * /` need **at least two** arguments here: `(+ 72)` is an error ("expects at least 2 arguments").
- `-` with one argument negates: `(- 144)` ⇒ `-144`, `(- -21)` ⇒ `21`.
- Many arguments work left to right: `(- 10 1 2)` ⇒ 10 − 1 − 2 = `7`; `(/ 720 5 4 3 2 1)` ⇒ `6`.

TRAP

An open parenthesis always means "apply the thing after me". `(5)` and `((+ 1 2))` are errors: "expected a function after the open parenthesis". Never add brackets "for safety" the way you might in maths.

L01 · 3

Substitution steps

Evaluation is rewriting. Each step (written ⇒) replaces **one** subexpression with its value, and the result is always a valid, simpler program. You stop when you reach a value.

STEPS: ONE SUBSTITUTION STEP

1. Scan from the left for the first subexpression that has **no parentheses inside it**, so every argument is already a value.
2. Replace that whole subexpression with its value. That is one step, even if the operator has many arguments: `(+ 11 12 13)` ⇒ `36`.
3. Copy everything else unchanged. Repeat.

```
(- (+ 9 (* (/ 6 (+ 1 1)) 3)) 2)
⇒ (- (+ 9 (* (/ 6 2) 3)) 2)
⇒ (- (+ 9 (* 3 3)) 2)
⇒ (- (+ 9 9) 2)
⇒ (- 18 2)
⇒ 16
```

TRAP

Only one rewrite per step, leftmost first. `(+ (* 2 3) (* 4 5))` ⇒ `(+ 6 (* 4 5))` ⇒ `(+ 6 20)` ⇒ `26`. Writing `(+ 6 20)` as the first step loses marks on a "show every step" question.

SHORTCUT

Values never step: numbers, `true` / `false`, symbols, characters, strings, `empty`, and lists made only of values like `(cons 1 empty)` or `(list 1 2)`.

L01 · 4

Defining constants with `define`

```
(define hours-in-day 24)
(define minutes-in-day (* hours-in-day 60))   ; evaluated once: 1440
```

- `(define name expr)` evaluates `expr` to a value first, then **binds** that value to `name`. In Racket these are called **constants**, not variables, because they never change.
- A name must be defined **before** it is used (error: "x is used here before its definition").
- A name can't be **re-defined** (error: "this name was defined previously and cannot be re-defined"). This includes using one name for both a constant and a function.
- Give meaningful numbers a name instead of leaving "magic numbers" in code (`freezing`, `feet-in-mile`).
- Identifier rules (L02): letters, digits and characters like `- _ . ? = !`; no spaces, brackets or quote marks; at least one non-digit. `2d`, `x-ray`, `?` and `hello!` are all valid names.

L01 · 5

Arithmetic operators, `quotient`, `remainder`, `sqr`, `expt`, `max`, `min`

OPERATOR	MEANING	EXAMPLE
`quotient`	Integer division, drops the fractional part (toward zero). Ints only.	`(quotient 43 7)` ⇒ 6, `(quotient -7 2)` ⇒ -3
`remainder`	What is left over. Sign follows the first argument. Ints only.	`(remainder 43 7)` ⇒ 1, `(remainder -7 2)` ⇒ -1
`sqr`	x^2	`(sqr 12)` ⇒ 144
`expt`	x^y	`(expt 3 4)` ⇒ 81, `(expt 2 -1)` ⇒ 1/2
`max` / `min`	Largest / smallest of **one or more** numbers	`(max 4 8 -9 3)` ⇒ 8, `(min 9)` ⇒ 9

Relationship to remember: `n = d · (quotient n d) + (remainder n d)`. `quotient` and `remainder` on a Rat such as `3/2` is an error ("expects an integer").

SHORTCUT: DIGIT AND DIVISIBILITY TRICKS

- Last digit of n: `(remainder n 10)`. Drop the last digit: `(quotient n 10)`. Tens digit: `(remainder (quotient n 10) 10)`.
- Is n divisible by d: `(zero? (remainder n d))`. Even: `(zero? (remainder n 2))`, or `even?` from L07.
- Minutes to hours and minutes: `(quotient m 60)` and `(remainder m 60)`.
- Clamp x into [lo, hi]: `(max lo (min hi x))`.

TRAP

You may *compute* with `(quotient n 10)`, but you may not *recurse* on it. The Rules for Recursion only allow `(sub1 n)` for a Nat (see L04).

L01 · 6

Comparisons and Bool expressions

- `(< x 5)` is a **question** ("is x less than 5?") that evaluates to `true` or `false`. It does not state a fact about x like it does in maths.
- `< > = <= >=` compare **numbers only**. `(= 'a 'a)` is an error. Each type has its own equality: `symbol=?` (L03), `char=?` and `string=?` (L12).
- Bool has exactly two values: `true` and `false`.
- `and` is true when every argument is true; `or` is false when every argument is false; `not` flips one Bool. `and` / `or` need at least two arguments.
- Arguments must be Bools: `(and 1 true)` is an error ("question result is not true or false").

Short-circuit evaluation

`and` stops at the first `false` and `or` stops at the first `true`; the remaining arguments are **never evaluated**.

```
(and (or (< 2 3) (> 1 5)) (not (< 10 15)) (<= 5 5))
⇒ (and (or true (> 1 5)) (not (< 10 15)) (<= 5 5))
⇒ (and true (not (< 10 15)) (<= 5 5))      ; or hit a true: done
⇒ (and true (not true) (<= 5 5))
⇒ (and true false (<= 5 5))
⇒ false                                    ; (<= 5 5) never evaluated
```

SHORTCUT: THE GUARD PATTERN

Put the safety check first so the risky part only runs when it is safe: `(and (cons? lst) (symbol=? (first lst) 'a))` never calls `first` on an empty list; `(and (not (zero? d)) (> (/ n d) 1))` never divides by zero.

De Morgan's laws

```
(not (and A B))  ≡  (or (not A) (not B))
(not (or A B))   ≡  (and (not A) (not B))
;; then push not into comparisons:
(not (< a b)) ≡ (>= a b)      (not (> a b)) ≡ (<= a b)
```

ON THE EXAM

Simplify `(not (or (< x 3) (> x 9)))` → `(and (>= x 3) (<= x 9))`. Or show each substitution step of a Bool expression, stopping exactly where short-circuiting kicks in.

SHORTCUT

Comparisons accept more than two arguments: `(<= 0 x 10)` means $0 \le x \le 10$. The slides only show the two-argument form, so `(and (<= 0 x) (<= x 10))` is the safe way to write it on an exam.

;; L02

Functions

User-defined functions, the formal substitution rules that give programs their meaning, and inexact numbers.

YOU SHOULD KNOW

- Defining functions with `define`, and applying them
- Predicates and the `?` naming convention
- The formal substitution rules for built-ins, Bool operators and user-defined functions
- Inexact numbers and Nat vs Int vs Rat vs Num
- Math functions `abs acos asin atan cos exp log sin sqrt tan`, constants `e pi`, predicates `number?` and `inexact?`
- How to format an assignment submission

L02 · 1

Defining functions

```
;; f(x) = x² + 3x + 4
(define (f x) (+ (* x x) (* 3 x) 4))
;; g(x, y) = x² + 6xy + y² + 9x - 3y - 100
(define (g x y)
  (+ (sqr x) (* 6 x y) (sqr y) (* 9 x) (- (* 3 y)) -100))
(check-expect (f 2) 14)
(check-expect (g 2 3) -42)
```

A definition has three parts: a **name**, a list of **parameters**, and **exactly one body expression**. Shape: `(define (name p1 p2 …) body)`.

- Renaming parameters consistently changes nothing: `(define (f x) (* x x))` behaves like `(define (f z) (* z z))`.
- Different functions may reuse the same parameter names.
- **Parameter order matters**: `(define (h x y) (- x y))` and `(define (h y x) (- x y))` are different functions.

TRAP

Calling with the wrong number of arguments is an error. A function name without parentheses around it is not a call; in our language, using a function name on its own is an error.

L02 · 2

Applying user-defined functions

STEPS

1. Evaluate the arguments, left to right, until **every** argument is a value. Don't touch the function yet.
2. In one step, replace the whole application with the function's body, substituting each argument value for **every** occurrence of its parameter.
3. Keep evaluating the body.

```
(define (h x y) (+ x x x y))
(h (+ 4 6) 9)
⇒ (h 10 9)              ; argument first
⇒ (+ 10 10 10 9)        ; ALL three x's replaced in a single step
⇒ 39
```

When several applications are ready, pick the **leftmost**, and only apply a user-defined function once all its arguments are simple values.

L02 · 3

Predicates and the `?` convention

A **predicate** is a function that produces a Bool. By convention its name ends in `?`.
Predicates are often built from other predicates.

```
(define (cold? t) (< t 8))
(define (cool? t) (and (< t 16) (not (cold? t))))
(define (hot? t) (> t 30))
(define (warm? t) (not (or (cold? t) (cool? t) (hot? t))))
(check-expect (cool? 12) true)
(check-expect (warm? 20) true)
(check-expect (warm? 32) false)
```

SHORTCUT: NO COND NEEDED

A predicate body is usually one Bool expression. Don't wrap it in a `cond` that produces `true` / `false`.

```
;; clunky
(define (teen? age) (cond [(and (>= age 13) (<= age 19)) true]
                          [else false]))
;; better
(define (teen? age) (and (>= age 13) (<= age 19)))
```

Built-in predicates by the midterm: `zero? number? boolean? inexact? symbol? empty? cons? list? even? odd? char? string?`.

L02 · 4

The formal substitution rules

These rules are the **semantics** (meaning) of a program. **Syntax** is only its form: `(+ 5 2)` is a syntax error; `(+ 5 2) ⇒ 7` is semantics.

RULE	PATTERN	CONDITION
0 · built-in	`(f v1 … vn) ⇒ v`	f is built-in (not and/or/not), all vi are values, v is the answer
1 · and	`(and true … true false …) ⇒ false` `(and true … true) ⇒ true`	Stop at the first false
1 · or	`(or false … false true …) ⇒ true` `(or false … false) ⇒ false`	Stop at the first true
1 · not	`(not true) ⇒ false`, `(not false) ⇒ true`	
2 · user function	`(f v1 … vn) ⇒ e'`	`(define (f x1 … xn) e)` appears above; e' is e with every xi replaced by vi
Constants	`x ⇒ v`	`(define x v)` appears above. The right side is simplified to a value first.
3 · cond (L03)	see L03	

```
(cool? 5)
⇒ (and (< 5 16) (not (cold? 5)))   ; rule 2
⇒ (and true (not (cold? 5)))       ; rule 0
⇒ (and true (not (< 5 8)))         ; rule 2
⇒ (and true (not true))            ; rule 0
⇒ (and true false)                 ; rule 1 (not)
⇒ false                            ; rule 1 (and)
```

```
(define x 3)
(define y 4)
(+ (* x y) (* 4 y))
⇒ (+ (* 3 y) (* 4 y))      ; each constant lookup is its own step
⇒ (+ (* 3 4) (* 4 y))
⇒ (+ 12 (* 4 y))
⇒ (+ 12 (* 4 4))
⇒ (+ 12 16)
⇒ 28
```

ON THE EXAM

"Show the substitution steps" or "write the next step". Label each step with its rule if asked. The common slips: doing two rewrites in one step, applying a function before its arguments are values, or forgetting that replacing a constant's name is a step.

L02 · 5

Inexact numbers, Num, and the math library

- Irrational numbers can't be stored exactly, so Racket uses hardware **floating point**, shown with a `#i` prefix: `(sqrt 2)` ⇒ `#i1.4142135623730951`. About 16 significant digits; range roughly $\pm 10^{308}$.
- **Num** means any number: Nat, Int, Rat or inexact. Nat ⊆ Int ⊆ Rat ⊆ Num.
- Inexact values come from `sqrt` of non-perfect squares, trig, `log`, `exp`, `pi`, `e`. `(sqrt 4)` is still exactly `2`.
- Inexact is contagious: `(+ 1 #i0.5)` ⇒ `#i1.5`.
- `number?` asks "is this any number" (not a Bool, symbol, list...). `inexact?` asks "is this number inexact". `boolean?` asks "is this a Bool".

FUNCTION	MEANING
`abs`	absolute value
`sqrt`	square root
`log`	**natural** log (base e). Base b: `(/ (log x) (log b))`

`exp`	e^x
`sin cos tan`	take **radians**. Degrees to radians: `(* deg (/ pi 180))`
`asin acos atan`	produce radians
`pi e`	constants (inexact)

TRAP: NEVER TEST INEXACT NUMBERS FOR EQUALITY

`(sin pi)` ⇒ `#i1.2246467991473532e-16`, so `(= (sin pi) 0)` is `false`. Compare with a tolerance instead: `(< (abs (- x y)) 0.0001)`. In tests, `check-expect` with an inexact result is an **error** ("cannot be an inexact number"); use `check-within`.

L02 · 6

Comments and submission format

- `;` starts a comment to the end of the line. Convention: `;;` for a whole-line comment, `;` after code on the same line.
- File header comment: name, student ID, term, assignment and problem number. Label each part (`;; Question 4. Part a.`) in assignment order.
- Every function gets a purpose comment above it. From L03 on, also a contract and tests (see L03 · 7).
- Lines at most 102 characters.
- To comment out a block: Racket → Comment Out with Semicolons, or `#| … |#`. **Never** "Comment Out with a Box": it can't be marked.

;; L03

Conditional expressions

`cond`, symbols, testing and contracts. After this lecture every function you submit needs a purpose, a contract, the code and full-coverage tests.

YOU SHOULD KNOW

- Writing question/answer pairs with `cond` and `else`
- The substitution rules for `cond`, and stepping a conditional
- Producing and consuming symbols: Sym, `symbol?`, `symbol=?`
- Testing with `check-expect` and `check-within`; finding boundary cases from a `cond`
- Writing and reading contracts
- What an assignment submission must include

L03 · 1

Question/answer pairs with `cond` and `else`

```
;; Choose outerwear for a temperature in degrees C.
;; what-to-wear: Num -> (anyof 'jacket 'sweater 'shirt)
(define (what-to-wear t)
  (cond [(< t 8)  'jacket]
        [(< t 16) 'sweater]     ; already know t >= 8 here
        [else     'shirt]))
(check-expect (what-to-wear -34) 'jacket)
(check-expect (what-to-wear 7) 'jacket)
(check-expect (what-to-wear 8) 'sweater)
(check-expect (what-to-wear 15) 'sweater)
(check-expect (what-to-wear 16) 'shirt)
(check-expect (what-to-wear 38) 'shirt)
```

- Each `[question answer]` pair: the question must evaluate to a Bool; the answer can be any value. Square brackets are a readability convention (the stepper may show round ones).
- Questions are tried **top to bottom**. The first true question's answer is the value of the whole `cond`; nothing after it is evaluated.
- **Order matters.** Every question can assume all earlier questions were false, so don't repeat their conditions. Above, the second question doesn't need `(and (>= t 8) …)`.
- `else` catches everything left and must be last. If no question is true and there is no `else`: error "all question results were false".

TRAP: DON'T NEST ELSE (COND …)

`[else (cond [A x] [else y])]` should be flattened into the outer `cond` as `[A x] [else y]`. Markers and the submission system flag the nested version.

SHORTCUT: BOOL CONDS COLLAPSE

INSTEAD OF	WRITE
`(cond [A true] [else false])`	`A`
`(cond [A false] [else true])`	`(not A)`
`(cond [A B] [else false])`	`(and A B)`
`(cond [A true] [else B])`	`(or A B)`

L03 · 2

Substitution rules for `cond`, and stepping one

```
(cond [false e] …)  ⇒  (cond …)     ; drop a false pair
(cond [true e] …)   ⇒  e            ; take the answer, discard the rest
(cond [else e])     ⇒  e
```

STEPS

1. Evaluate only the **first** question, one substitution at a time, until it is `true` or `false`.
2. If `false`: one step deletes that pair. Go back to step 1 with the new first pair.
3. If `true` (or the pair is `else`): one step replaces the **whole cond** with that answer. Answers are never evaluated before they are chosen.

```
(define (sign x) (cond [(> x 0) 'pos] [(< x 0) 'neg] [else 'zero]))
(sign 0)
⇒ (cond [(> 0 0) 'pos] [(< 0 0) 'neg] [else 'zero])
⇒ (cond [false 'pos] [(< 0 0) 'neg] [else 'zero])
⇒ (cond [(< 0 0) 'neg] [else 'zero])
⇒ (cond [false 'neg] [else 'zero])
⇒ (cond [else 'zero])
⇒ 'zero
```

TRAP

Dropping a false pair is its own step, and so is the final `(cond [else e]) ⇒ e`. Answers stay unevaluated (with the parameter values substituted in) until their pair is chosen.

L03 · 3

Symbols: Sym, `symbol?`, `symbol=?`

- A symbol is a value written with a leading quote: `'jacket`, `'CS135`. It means something to *you*, not to Racket. Symbols are self-documenting labels, so you don't need constants like `(define jacket 0)`.
- No arithmetic on symbols; the only thing you do is compare them.
- `(symbol=? a b)`: both arguments **must** be symbols. `(symbol=? 'blue 42)` is an error.
- `(symbol? v)`: works on any value. `(symbol? 42)` ⇒ `false`.
- Symbols are case-sensitive: `(symbol=? 'Apple 'apple)` ⇒ `false`.

TRAP: MIXED DATA NEEDS A TYPE CHECK FIRST

When a value can be a number *or* a symbol (like a card Rank: 2–10 or `'jack` ...), check the type before comparing, or the comparison errors:

```
;; rank=?: Rank Rank -> Bool
(define (rank=? r0 r1)
  (or (and (symbol? r0) (symbol? r1) (symbol=? r0 r1))
      (and (number? r0) (number? r1) (= r0 r1))))
(check-expect (rank=? 'ace 'ace) true)
(check-expect (rank=? 2 2) true)
(check-expect (rank=? 'king 10) false)
```

Short-circuiting makes this safe: `symbol=?` only runs once both are known to be symbols.

L03 · 4

Testing with `check-expect` and `check-within`

- `(check-expect actual-expr expected-value)`: passes when they are equal. Use it for exact numbers, Bools, symbols, lists, strings.
- `(check-within actual-expr expected-value delta)`: passes when they differ by at most `delta`. Use it whenever the result is inexact: `(check-within (sqrt 2) 1.414 0.001)`.
- Tests can sit anywhere in the file, even above the function; they run after everything is defined.

- **Before** coding, write tests ("examples") worked out by hand: they show whether you understand the problem. **After** coding, add tests until coverage is complete.

L03 · 5

Boundary cases and full coverage

STEPS: TESTS FROM A COND

1. For each question except `else` : at least one test where it is true **and** one where it is false (and the earlier ones are false).
2. For each threshold, test **at** the threshold and just on either side. Thresholds 8 and 16 → test 7, 8, 15, 16.
3. Add extremes: −34, 38, very large, 0.
4. Run. Any code DrRacket highlights was never executed, so coverage is incomplete. Assignments require zero highlighting.

SHORTCUT: STANDARD TEST CHECKLIST

- Nat recursion: 0, 1, a larger n.
- Lists: `empty` ; one element; target at the front, in the middle, at the end; all match; none match; duplicates.
- Two lists: both empty, each one empty, different lengths.
- Sorted input: already sorted, reversed, with equal values.

L03 · 6

Contracts

A contract is a comment saying what types a function consumes and produces: `;; name: In1 In2 -> Out` .

- Types by the midterm: `Nat Int Rat Num Bool Sym Char Str Any` , `(listof X)` , `(anyof …)` , and names from your data definitions (`Point` , `Card` , `AL`).
- Use the most precise type that is true. Counting → `Nat` . Anything with division or `sqrt` → `Num` .
- `anyof` lists the exact values or types allowed: `Num -> (anyof 'jacket 'sweater 'shirt)` , `(anyof Num empty)` . `Num -> Sym` is also acceptable unless `anyof` is requested.
- Add a **Requires** line for limits the types can't express:

```
;; last: (listof Any) -> Any
;; Requires: lst is non-empty
;; insert: Num (listof Num) -> (listof Num)
;; Requires: lst is sorted in ascending order
```

L03 · 7

What a submission must include

1. **Purpose**: what the function produces, in terms of its parameter names.
2. **Contract** (plus Requires if needed).
3. The **function definition**.
4. **Tests** giving full coverage.

Default grading: purpose 5%, contract 5%, coverage 10%, correctness 80% (share of hidden tests passed). From L08: helper functions you invent don't lose marks for a missing purpose or contract, but still need test coverage.

Exact formats for every piece, with a full example file: Design recipe & style.

;; L04

Recursion on natural numbers

A function defined in terms of itself on a smaller argument. With big numbers, conditionals and recursion, the model is Turing complete: it can compute anything any computer can.

YOU SHOULD KNOW

- Using the design recipe to write functions
- Writing recursive functions that follow version 1 of the Rules for Recursion
- Using `zero?` and `sub1`
- Why those rules guarantee termination
- Measuring efficiency with the stepper

L04 · 1

The design recipe

STEPS

1. **Purpose**: understand the problem and write it down.
2. **Contract and header**: types, then `(define (name params) …)`.
3. **Examples**, worked out by hand, as `check-expect`s.
4. **Body**, starting from a template.
5. **More tests** for boundary cases and coverage.

```
;; (1) Compute n! (the product 1·2·…·n), with 0! = 1.
;; (2) factorial: Nat -> Nat
(define (factorial n)
  ;; (4) body from the Nat template
  (cond [(zero? n) 1]
        [else (* n (factorial (sub1 n)))]))
;; (3) examples, (5) boundaries
(check-expect (factorial 0) 1)
(check-expect (factorial 1) 1)
(check-expect (factorial 3) 6)
(check-expect (factorial 5) 120)
```

How to write each step, and the test rules: Design recipe & style.

L04 · 2

Writing recursive functions on a Nat

```
(define (nat-template n)
  (cond [(zero? n) …]                          ; base case
        […  …]                                 ; optional extra cases
        [else (… n … (nat-template (sub1 n)))]))
```

STEPS: FILLING THE TEMPLATE

1. Write the maths as a recurrence: f(0) = ?, f(n) = something using f(n−1). For sum-to: f(0) = 0, f(n) = n + f(n−1).
2. The base answer goes in `[(zero? n) …]`.
3. Trust that `(f (sub1 n))` already gives the right answer for n−1 (the "leap of faith"). Ask only how to turn that into the answer for n.
4. If some n need different handling (only multiples of 3 count, say), add a middle clause.

```
;; sum-to: Nat -> Nat        0 + 1 + … + n
(define (sum-to n)
  (cond [(zero? n) 0]
        [else (+ n (sum-to (sub1 n)))]))
(check-expect (sum-to 4) 10)
(check-expect (sum-to 0) 0)

;; power: Num Nat -> Num      b^n by repeated multiplication
(define (power b n)
  (cond [(zero? n) 1]
        [else (* b (power b (sub1 n)))]))   ; b never changes
(check-expect (power 2 10) 1024)
(check-expect (power 5 0) 1)

;; count-mult3: Nat -> Nat    how many of 1..n are divisible by 3
(define (count-mult3 n)
  (cond [(zero? n) 0]
        [(zero? (remainder n 3)) (add1 (count-mult3 (sub1 n)))]
        [else (count-mult3 (sub1 n))]))
(check-expect (count-mult3 10) 3)
(check-expect (count-mult3 2) 0)
```

SHORTCUT: PICKING THE BASE VALUE

It is the answer for 0, which is usually the identity of the combining operation: 0 for `+`, 1 for `*`, `empty` when building a list with `cons`, `true` for "all", `false` for "any".

L04 · 3

`zero?` and `sub1`

- `(zero? n)` is true only when n is 0. It is the termination test for Nat recursion.
- `(sub1 n)` is n − 1. It is the only allowed way to make a Nat argument smaller. (`add1` arrives in L06.)

L04 · 4

Rules for Recursion (v1) and why they guarantee termination

RULES, VERSION 1

1. Change **one** argument closer to termination when recurring. No other argument may change.
2. When recurring on a natural number, use `(sub1 n)` and test termination with `zero?` .

Why it terminates: n is a Nat, and each recursive call is on n − 1. A Nat can only be decreased by 1 a finite number of times before it reaches 0, where `zero?` stops the recursion.

The three properties CS135 cares about: **termination** (guaranteed by the rules), **correctness** (could be proved by induction; we use tests), **efficiency** (count of substitution steps).

TRAP

- Called with −1 or 2.5, the function never reaches 0 and runs forever. That is why the contract says `Nat` .
- Not allowed: recursing on `(- n 2)` , `(quotient n 10)` or `(add1 n)` , or testing termination with `(= n 1)` or `(<= n 0)` instead of `zero?` .
- "No other argument may change": `power` above passes `b` through unchanged. In v1 only one argument may move. (L09 allows several; L10 allows accumulators.)

L04 · 5

Measuring efficiency with the stepper

- Efficiency in CS135 = **number of substitution steps**. DrRacket's Stepper shows a counter like `1/64` : `(sum-to 10)` takes 64 steps.
- The counter counts every expression the stepper shows, the starting one included, so it is one more than the number of ⇒ arrows.

- `(sum-to n)` takes 6n + 4: four steps for the base case, six per level of recursion (expand the call, evaluate the question, drop the false pair, take `else`, `sub1`, and the final `+`).

L04 · extra

Scope (from L04.2)

- Two kinds of scope for now: **global** (top-level definitions) and **function** (parameters, visible only inside that body).
- The **smallest enclosing scope wins**: a parameter named `x` hides a global constant `x` inside that function.
- Two definitions of the same name in the same scope is an error.

```
(define x 3)
(define (f x y) (- x y))     ; this x is the parameter
(f 10 x)
⇒ (f 10 3)                   ; the argument x is the global one
⇒ (- 10 3)
⇒ 7
```

TRAP

Avoid naming a parameter `list`: inside that function it hides the built-in `list` (L08). Use `lst`, `lon` (list of numbers), `los` (list of symbols).

;; L05

Lists

The one composite data type in CS135. Fixed-size lists play the role that "structures" play elsewhere.

YOU SHOULD KNOW

- Creating and taking apart lists with `cons first rest empty empty? list? cons?`
- Writing data definitions and helper functions for composite data built from lists
- Applying the design recipe to functions that use composite data

L05 · 1

The list operations

OPERATION	CONSUMES → PRODUCES	NOTES
`empty`	the empty list	A value. An empty list is still a list.
`(cons v lst)`	Any, list → list	New list with v added to the **front**. The second argument must be a list: `(cons 1 2)` is an error.
`(first lst)`	non-empty list → Any	The first **element**. Error on `empty`.
`(rest lst)`	non-empty list → list	The list **without** its first element (always a list). Error on `empty`.
`(empty? v)`	Any → Bool	true only for `empty`
`(cons? v)`	Any → Bool	true only for a **non-empty** list
`(list? v)`	Any → Bool	true for any list, empty or not

```
(define groceries (cons 'milk (cons 'apple (cons 'bread empty))))
(check-expect (first groceries) 'milk)
(check-expect (rest groceries) (cons 'apple (cons 'bread empty)))
(check-expect (first (rest groceries)) 'apple)          ; 2nd element
(check-expect (first (rest (rest groceries))) 'bread)   ; 3rd element
(check-expect (rest (rest (rest groceries))) empty)
```

SHORTCUT: READING FIRST/REST CHAINS

Read from the inside out: each `rest` drops one element from the front, then `first` takes the front. `(first (rest (rest lst)))` = drop, drop, take = the 3rd element.

TRAP

- `(rest (cons 5 empty))` is `empty`, not 5. `rest` always produces a list.
- `cons` only adds at the front. Adding at the end needs recursion (`attach`, L08) or `append` (L09).
- `(cons (list 1 2) (list 3 4))` is `(list (list 1 2) 3 4)`, a list whose first element is a list. It does not join the lists.

L05 · 2

Data definitions and helper functions

A **data definition** names a composite type so contracts stay readable. Pair it with a **constructor** (`mk-`) and **accessors** (`get-`), then write every other function through those helpers instead of raw `first` / `rest`.

```
;; A Point is a (cons Num (cons Num empty))    ; (list Num Num) after L08
;; mk-point: Num Num -> Point
(define (mk-point x y) (cons x (cons y empty)))
;; get-x: Point -> Num
(define (get-x p) (first p))
;; get-y: Point -> Num
(define (get-y p) (first (rest p)))

;; Distance from a Point to the origin.
;; distance-to-origin: Point -> Num
(define (distance-to-origin p)
  (sqrt (+ (sqr (get-x p)) (sqr (get-y p)))))
(check-expect (distance-to-origin (mk-point -3 4)) 5)
(check-within (distance-to-origin (mk-point 3 -1)) 3.1622 0.001)
(check-expect (distance-to-origin (mk-point 0 0)) 0)
```

Data definitions can also name a set of symbols: `;; An Outerwear is (anyof 'jacket 'sweater 'shirt)`, then `what-to-wear: Num -> Outerwear`.

STEPS: NEW COMPOSITE DATA ON AN EXAM

1. Write (or read carefully) the data definition: which position holds which field, and their types.
2. Write `mk-` and `get-` helpers. Accessor for position k: first, then `first (rest …)`, etc. (or `second` / `third` from L07).
3. Use the type name in contracts: `distance: Point Point -> Num`.
4. Build test data with the constructor.

L05 · 3

Design recipe with composite data: playing cards

```
;; A Suit is (anyof 'spade 'heart 'diamond 'club)
;; A Rank is (anyof 2 3 4 5 6 7 8 9 10 'jack 'queen 'king 'ace)
;; A Card is a (cons Suit (cons Rank empty))
(define (mk-card suit rank) (cons suit (cons rank empty)))
(define (get-suit card) (first card))
(define (get-rank card) (first (rest card)))

;; Is a Card a Jack, Queen or King?   (uses rank=? from L03)
;; face-card?: Card -> Bool
(define (face-card? card)
  (or (rank=? (get-rank card) 'jack)
      (rank=? (get-rank card) 'queen)
      (rank=? (get-rank card) 'king)))
(check-expect (face-card? (mk-card 'heart 6)) false)
(check-expect (face-card? (mk-card 'club 'king)) true)
(check-expect (face-card? (mk-card 'diamond 'ace)) false)

;; same-suit?: Card Card -> Bool
(define (same-suit? c0 c1) (symbol=? (get-suit c0) (get-suit c1)))
(check-expect (same-suit? (mk-card 'heart 2) (mk-card 'heart 'ace)) true)
(check-expect (same-suit? (mk-card 'club 2) (mk-card 'heart 2)) false)
```

TRAP

`(symbol=? (get-rank card) 'king)` crashes when the rank is a number. That is why `rank=?` exists. Suits are always symbols, so `symbol=?` is safe for them.

;; L06

Recursion on a list

A list is defined recursively, so functions on lists are recursive in the same shape. This lecture is about folding a list down to one value, and predicates over lists.

YOU SHOULD KNOW

- Writing recursive functions that fold lists, using the design recipe, the template and the Rules for Recursion (v2)
- Writing predicates over lists the same way
- You can't use `member?` or `equal?`

L06 · 1

The list data definition, template and the four questions

```
;; A (listof X) is one of:
;; * empty
;; * (cons X (listof X))

(define (listof-X-template lox)
  (cond [(empty? lox) …]
        […  …]                                    ; optional cases on (first lox)
        [else (… (first lox) … (listof-X-template (rest lox)) …)]))
```

Replace X with a real type (`Num` , `Sym` , `Point`), or use `Any` when the function doesn't care what the elements are.

STEPS: THE FOUR QUESTIONS

1. What should the function produce for the **empty** list (the base case)?
2. What should it do with the **first** element of a non-empty list?
3. What does the recursive call on the **rest** produce? (Assume it's correct.)
4. How do I **combine** 2 and 3 into the answer for the whole list?

RULES FOR RECURSION, VERSION 2

1. Change one argument closer to termination; no other argument may change.
2. On a natural number: `(sub1 n)` , test with `zero?` .
3. On a list: `(rest lst)` , test with `empty?` .

The empty case always comes first in the `cond` , so `first` / `rest` are never applied to `empty` . Extra parameters that don't drive the recursion (like a target symbol) are passed along unchanged.

TRAP

Recursing in a **question** of a `cond` is almost always a mistake (and leads to the exponential blow-up in L08). Recursing in two different **answers** is fine, because only one answer runs.

L06 · 2

Folding a list

Folding (or reducing) boils a whole list down to one value: a count, a sum, a length, a Bool. You pick a base value for `empty` and a way to combine each element with the answer for the rest.

```
;; count-symbol: Sym (listof Sym) -> Nat
(define (count-symbol target los)
  (cond [(empty? los) 0]
        [(symbol=? target (first los))
         (add1 (count-symbol target (rest los)))]   ; count it: +1
        [else (count-symbol target (rest los))]))     ; ignore it: +0
(check-expect (count-symbol 'apple empty) 0)
(check-expect (count-symbol 'apple (cons 'apple (cons 'eggs (cons 'apple empty)))) 2)
(check-expect (count-symbol 'fish (cons 'apple empty)) 0)

;; len: (listof Any) -> Nat      (the built-in is length)
(define (len lst)
  (cond [(empty? lst) 0]
        [else (add1 (len (rest lst)))]))
(check-expect (len empty) 0)
(check-expect (len (cons 1 (cons 'a (cons true empty)))) 3)

;; sum: (listof Num) -> Num
(define (sum lon)
  (cond [(empty? lon) 0]
        [else (+ (first lon) (sum (rest lon)))]))
(check-expect (sum (cons 3 (cons -1 (cons 10 empty)))) 12)

;; count-above: Num (listof Num) -> Nat
(define (count-above k lon)
  (cond [(empty? lon) 0]
        [(> (first lon) k) (add1 (count-above k (rest lon)))]
        [else (count-above k (rest lon))]))
(check-expect (count-above 2 (cons 1 (cons 5 (cons 3 empty)))) 2)
```

FOLD	BASE FOR EMPTY	COMBINE
count / length	`0`	`(add1 recur)` or just `recur`
sum	`0`	`(+ (first lst) recur)`

product	`1`	`(* (first lst) recur)`
all-?	`true`	false on the first failure, else recur
any-? / contains?	`false`	true on the first success, else recur
max / min	no base for empty: use the non-empty template (L07)	`(max (first lst) recur)`

TRAP

Built-in `length` is allowed from L06 on, but you can't name your own function `length` (that's why the slides call it `len`). And a product with base `0` is always 0.

L06 · 3

Predicates over lists

Two mirror-image shapes. Learn which base value goes with which.

"Is it true for ALL?"

```
;; all-positive?: (listof Num) -> Bool
(define (all-positive? lon)
  (cond [(empty? lon) true]          ;
nothing fails
        [(<= (first lon) 0) false]   ; one
failure: stop
        [else (all-positive? (rest lon))]))
(check-expect (all-positive? empty) true)
(check-expect (all-positive? (cons 1 (cons
-2 (cons 3 empty)))) false)
(check-expect (all-positive? (cons 1 (cons 2
empty))) true)
```

"Is it true for ANY?"

```
;; contains-symbol?: Sym (listof Sym) ->
Bool
(define (contains-symbol? target los)
  (cond [(empty? los) false]
; nothing found
        [(symbol=? target (first los)) true]
; found: stop
        [else (contains-symbol? target (rest
los))]))
(check-expect (contains-symbol? 'b empty)
false)
(check-expect (contains-symbol? 'b (cons 'a
(cons 'b empty))) true)
(check-expect (contains-symbol? 'z (cons 'a
empty)) false)
```

SHORTCUT

"All" on an empty list is `true` (nothing breaks the rule). "Any/contains" on an empty list is `false` (nothing to find). Both short-circuit as soon as the answer is known.

L06 · 4

No `member?`, no `equal?`: write your own

`equal?` compares anything with anything, deeply. The course bans it (and every built-in that uses it, like `member?`) so you always know the types you're comparing. If the docs say a function uses `equal?`, you can't use it.

```
;; contains?: (anyof Sym Num) (listof (anyof Sym Num)) -> Bool
(define (same-item? a b)
  (or (and (symbol? a) (symbol? b) (symbol=? a b))
      (and (number? a) (number? b) (= a b))))
(define (contains? target lst)
  (cond [(empty? lst) false]
        [(same-item? target (first lst)) true]
        [else (contains? target (rest lst))]))
(check-expect (contains? 3 (cons 'a (cons 3 empty))) true)
(check-expect (contains? 'a (cons 3 (cons 4 empty))) false)
```

COMPARING	USE
Num	`=` (or a tolerance for inexact)
Sym	`symbol=?`
Char / Str (L12)	`char=?` / `string=?`
Mixed	check the type with `number?` / `symbol?` first

;; L07

Producing a list

The other two list idioms, filter and map, plus ordered lists and functions on non-empty lists.

YOU SHOULD KNOW

- Filtering lists
- Transforming each element with the map idiom
- Checking that a list has an ordering property such as ascending
- Inserting into an ordered list
- The built-ins `second` and `third`
- Finding the last element of a list

L07 · 1

Filtering a list

A **filter** produces the sublist of elements that pass a test, in the original order. The whole trick: **to delete an element, don't `cons` it**; just return the recursive result. To keep it, `cons` it onto the recursive result.

```
;; Remove every occurrence of target.
;; eat-symbol: Sym (listof Sym) -> (listof Sym)
(define (eat-symbol target los)
  (cond [(empty? los) empty]
        [(symbol=? target (first los))
         (eat-symbol target (rest los))]                    ; drop: no cons
        [else (cons (first los) (eat-symbol target (rest los)))]))  ; keep
(check-expect (eat-symbol 'apple (cons 'apple (cons 'eggs (cons 'apple empty))))
              (cons 'eggs empty))
(check-expect (eat-symbol 'apple empty) empty)

;; Keep only even numbers.
;; keep-even: (listof Int) -> (listof Int)
(define (keep-even loi)
  (cond [(empty? loi) empty]
        [(odd? (first loi)) (keep-even (rest loi))]
        [else (cons (first loi) (keep-even (rest loi)))]))
(check-expect (keep-even (cons 4 (cons -3 (cons -8 (cons 1 empty)))))
              (cons 4 (cons -8 empty)))
```

Variation: remove only the first occurrence

```
;; remove-first: Sym (listof Sym) -> (listof Sym)
(define (remove-first target los)
  (cond [(empty? los) empty]
        [(symbol=? target (first los)) (rest los)]   ; found: stop, keep the rest as is
        [else (cons (first los) (remove-first target (rest los)))]))
(check-expect (remove-first 'a (cons 'b (cons 'a (cons 'a empty))))
              (cons 'b (cons 'a empty)))
```

SHORTCUT

`eat-symbol` (remove all) recurses after a match; `remove-first` produces `(rest los)` and stops. The same "stop early" move appears in `insert` and in dictionary `delete` (L12).

TRAP

Forgetting `(cons (first lst) …)` in the keep branch silently deletes everything. Consing in the drop branch deletes nothing. Base case for a list-producing function is `empty`, not `0`.

L07 · 2

Mapping (transforming) a list

A **map** produces a list of the **same length**, with a transformation applied to every element. Every non-empty branch conses *something*.

```racket
;; add1-list: (listof Num) -> (listof Num)
(define (add1-list lon)
  (cond [(empty? lon) empty]
        [else (cons (add1 (first lon)) (add1-list (rest lon)))]))
(check-expect (add1-list (cons 10 (cons -6 empty))) (cons 11 (cons -5 empty)))
(check-expect (add1-list empty) empty)

;; A conditional map: replace one symbol with another.
;; replace: Sym Sym (listof Sym) -> (listof Sym)
(define (replace old new los)
  (cond [(empty? los) empty]
        [(symbol=? old (first los)) (cons new (replace old new (rest los)))]
        [else (cons (first los) (replace old new (rest los)))]))
(check-expect (replace 'apple 'orange (cons 'apple (cons 'milk empty)))
              (cons 'orange (cons 'milk empty)))

;; A map with an extra unchanging parameter.
;; scale: Num (listof Num) -> (listof Num)
(define (scale k lon)
  (cond [(empty? lon) empty]
        [else (cons (* k (first lon)) (scale k (rest lon)))]))
(check-expect (scale 3 (cons 1 (cons 2 empty))) (cons 3 (cons 6 empty)))
```

SHORTCUT: TELL THE THREE IDIOMS APART BY THE OUTPUT

- One value out (number, Bool) → **fold**.
- Shorter-or-equal list, same elements → **filter**.
- Same-length list, changed elements → **map**.

L07 · 3

Checking an ordering property (ascending?)

A property about **neighbours** needs two elements at a time, so there are two base cases: an empty list and a one-element list are both trivially ordered.

```racket
;; ascending?: (listof Num) -> Bool
(define (ascending? lon)
  (cond [(or (empty? lon) (empty? (rest lon))) true]
        [(> (first lon) (first (rest lon))) false]   ; neighbours out of order
        [else (ascending? (rest lon))]))             ; (rest lon), NOT (rest (rest lon))
(check-expect (ascending? empty) true)
(check-expect (ascending? (cons -10 empty)) true)
(check-expect (ascending? (cons 10 (cons 20 (cons 30 empty)))) true)
(check-expect (ascending? (cons 1 (cons 1 empty))) true)
(check-expect (ascending? (cons 3 (cons 1 (cons 2 empty)))) false)
```

- Non-strict ascending (equal allowed): fail on `>`. **Strictly** increasing: fail on `>=`. Descending: flip the comparison.
- The second base case is what makes `(first (rest lon))` safe.

TRAP

Recursing on `(rest (rest lon))` skips the comparison between the 2nd and 3rd elements: `(list 1 5 2 6)` would wrongly pass. Comparing `(first lon)` with `(rest lon)` compares a number with a list and errors.

SHORTCUT

Same shape for any adjacent-pair question: "no two neighbours equal", "signs alternate", "every step goes up by exactly 1", "sum of adjacent pairs".

L07 · 4

Inserting into an ordered list

```
;; Insert n into its place in a sorted list.
;; insert: Num (listof Num) -> (listof Num)
;; Requires: lon is sorted in ascending order
(define (insert n lon)
  (cond [(empty? lon) (cons n empty)]                  ; end of list: n goes last
        [(< n (first lon)) (cons n lon)]                ; found its spot: stop
        [else (cons (first lon) (insert n (rest lon)))])) ; keep looking
(check-expect (insert -1 empty) (cons -1 empty))
(check-expect (insert 0 (cons 1 (cons 2 empty))) (cons 0 (cons 1 (cons 2 empty))))
(check-expect (insert 3 (cons 1 (cons 2 (cons 4 empty)))) (cons 1 (cons 2 (cons 3 (cons 4
empty)))))
(check-expect (insert 5 (cons 1 (cons 2 empty))) (cons 1 (cons 2 (cons 5 empty))))
(check-expect (insert 2 (cons 1 (cons 2 empty))) (cons 1 (cons 2 (cons 2 empty))))
```

THE THREE CASES

1. Empty list: the result is the one-element list `(cons n empty)`, not `empty`.
2. n belongs before the first element: `(cons n lon)`. The rest is already sorted, so stop recursing.
3. Otherwise keep the first element and insert into the rest.

TRAP

Losing `(first lon)` in case 3, returning `empty` in case 1, or recursing again after inserting (duplicating n). Test front, middle, end, empty, and a duplicate value.

L07 · 5

`second` and `third`

- `(second lst)` = `(first (rest lst))`, needs length ≥ 2. `(third lst)` = `(first (rest (rest lst)))`, needs length ≥ 3. Too short is an error.
- `fourth` through `eighth` exist but are **not allowed**. Needing them means there's a simpler approach (probably recursion).
- Great for fixed-size data: `(define (get-y p) (second p))`.

L07 · 6

The last element: the non-empty list template

```
;; last: (listof Any) -> Any
;; Requires: lst is non-empty
(define (last lst)
  (cond [(empty? (rest lst)) (first lst)]   ; one element left: it's the last
        [else (last (rest lst))]))
(check-expect (last (cons 7 empty)) 7)
(check-expect (last (cons 1 (cons 2 (cons 3 empty)))) 3)
```

When the function makes no sense on `empty` (last, largest, smallest), the base case is `(empty? (rest lst))`, which the Rules allow, and the contract gets a **Requires: non-empty** line.

```
(define (nonempty-template lst)
  (cond [(empty? (rest lst)) (… (first lst) …)]
        [else (… (first lst) … (nonempty-template (rest lst)) …)]))
```

;; L08

Efficiency

List abbreviations, then counting substitution steps to classify functions as constant, linear, quadratic or exponential.

YOU SHOULD KNOW

- Writing lists with list abbreviations
- Measuring efficiency with the stepper
- Sorting with insertion sort
- The difference between linear, quadratic and exponential time
- Recognizing and fixing exponential "blow up"

L08 · 1

List abbreviations

- `(list v1 v2 … vn)` is shorthand for `(cons v1 (cons v2 (… (cons vn empty))))`. Set the language to **Beginning Student with List Abbreviations** (needed for the step counts below to match).
- Results now print as `(list …)`.

```
(check-expect (list 4 3 2 1) (cons 4 (cons 3 (cons 2 (cons 1 empty)))))
(check-expect (cons 0 (list 1 2)) (list 0 1 2))
(check-expect (list) empty)
(check-expect (length (list empty)) 1)              ; a list holding one empty list
(check-expect (length (list (list 1 2) 3)) 2)       ; nested list counts as one element
```

TRAP

`(list empty)` is not empty: it has one element. `(list (list 1 2))` has one element, which is itself a list.

L08 · 2

Measuring efficiency with the stepper

Run the function on lists of length 0, 1, 2, 3, … and record the step counts. Then fit a formula:

STEPS: FROM A TABLE OF STEP COUNTS TO A FORMULA

1. Take differences between consecutive counts. **Constant first differences d** → linear: f(n) = d·n + f(0).
2. If the first differences grow by a constant amount **s** (constant second differences) → quadratic: a = s/2, c = f(0), b = f(1) − a − c.
3. If each count is roughly **double** the previous → exponential.

FUNCTION	N = 0, 1, 2, 3, 4	FORMULA	CLASS
`len`	4, 10, 16, 22, 28	6n + 4	linear
`add1-list`	4, 12, 20, 28, 36	8n + 4	linear
`attach` (to end)	5, 12, 19, 26, 33	7n + 5	linear
`sort` (reversed input)	4, 14, 34, 64, 104	$5n^2 + 5n + 4$	quadratic
`largest-slow` (ascending input)	–, 6, 21, 51, 111	≈ doubles	exponential

Check for `sort`: differences 10, 20, 30, 40 → s = 10 → a = 5; c = 4; b = 14 − 5 − 4 = 5.

- We care about the **worst case**: the input of size n that takes the most steps (for insertion sort, a list in reverse order).
- Built-ins take one stepper step, but treat `length`, `append` and `reverse` as **linear**, as if you'd written them yourself. All other allowed built-ins are constant time.

L08 · 3

Insertion sort

Recursive idea: sort the rest of the list, then insert the first element into its place. It reuses `insert` from L07 as a helper.

```
;; sort: (listof Num) -> (listof Num)
(define (sort lon)
  (cond [(empty? lon) empty]
        [else (insert (first lon) (sort (rest lon)))]))
(check-expect (sort empty) empty)
(check-expect (sort (list 100)) (list 100))
(check-expect (sort (list 5 -10 1 14 3 1 8)) (list -10 1 1 3 5 8 14))
(check-expect (sort (list 3 2 1)) (list 1 2 3))
```

It follows the Rules (one recursive call on `(rest lon)`), but it's **quadratic**: n recursive calls, and each `insert` may walk the whole sorted rest. Already-sorted input is fastest (each insert stops immediately); reversed input is the worst case.

SHORTCUT

"Solve the rest, then fold the first element in with a helper" is a general recipe. Insertion sort, the slow `rev` below, and `subsets` in L11 all have this shape.

L08 · 4

Linear vs quadratic vs exponential

CLASS	STEPS	TYPICAL CAUSE	BIG-O
Constant	fixed, whatever n is	no recursion; constant-time built-ins (`first` , `cons` , `+` ...)	O(1)
Linear	an + b	one recursive call per branch on `rest` / `sub1` , constant work per call	O(n)
Quadratic	$an^2 + bn + c$	a **linear** helper applied in every recursive call, usually to the result of the recursion	$O(n^2)$
Exponential	$\approx 2^n$	two recursive calls on the same `(rest lst)` in one evaluation	$O(2^n)$

```
;; attach: Any (listof Any) -> (listof Any)     linear: add to the end
(define (attach x lst)
  (cond [(empty? lst) (list x)]
        [else (cons (first lst) (attach x (rest lst)))]))
(check-expect (attach -1 (list 1 2)) (list 1 2 -1))

;; rev-slow: (listof Any) -> (listof Any)      quadratic: linear attach inside
(define (rev-slow lst)
  (cond [(empty? lst) empty]
        [else (attach (first lst) (rev-slow (rest lst)))]))
(check-expect (rev-slow (list 1 2 3)) (list 3 2 1))

;; countdown: Nat -> (listof Nat)               linear, on a Nat
(define (countdown n)
  (cond [(zero? n) empty]
        [else (cons n (countdown (sub1 n)))]))
(check-expect (countdown 3) (list 3 2 1))
```

TRAP: HIDDEN QUADRATIC

Calling `length` , `append` , `reverse` or your own list-walking helper inside every recursive step turns a linear function quadratic. Sometimes that's fine (it's not exponential), but `reverse` inside a recursive function is a red flag.

Practical rule from the slides: if your code handles a list of a few hundred elements in under ten seconds, it's efficient enough. Only exponential blow-up is a real problem.

L08 · 5

Recognizing and fixing exponential blow-up

```
;; BAD: (largest-slow (rest lst)) can be evaluated twice
(define (largest-slow lst)
  (cond [(empty? (rest lst)) (first lst)]
        [(> (first lst) (largest-slow (rest lst))) (first lst)]   ; call #1, in a question
        [else (largest-slow (rest lst))]))                        ; call #2
```

When the first element isn't the biggest, both calls run on the same rest, and each of those makes two calls, and so on: up to $2^n - 1$ calls. On ascending input of length 25 it practically never finishes.

STEPS: THE FIX

1. Spot the smell: the same recursive call written twice (often once in a question and once in an answer).
2. Make the recursive call **once** and hand its result to a helper (or a built-in like `max`) as an argument. Inside the helper it's a plain value, so using it twice is free.

```
;; larger: Num Num -> Num
(define (larger a b)
  (cond [(> a b) a]
        [else b]))
;; largest: (listof Num) -> Num
;; Requires: lon is non-empty
(define (largest lon)
  (cond [(empty? (rest lon)) (first lon)]
        [else (larger (first lon) (largest (rest lon)))]))   ; one call: linear
(check-expect (largest (list 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25))
25)
(check-expect (largest (list 9 -2 4)) 9)
;; equivalently: (max (first lon) (largest (rest lon)))
```

TRAP

Two recursive calls in **different answers** of a `cond` are fine: only one answer ever runs. The problem is two calls on the same input within one evaluation.

;; L09

Recursion on multiple arguments

Recursing on a number and a list together, the final Rules for Recursion, appending, and the three ways to recurse on two lists.

YOU SHOULD KNOW

- Recursion on a natural number and a list
- The final Rules for Recursion (use these unless told otherwise)
- Appending one list to another
- Recursion on two lists, including the "lockstep" pattern

L09 · 1

Recursion on a Nat and a list

Key observation for indexing: the item at position 2 of `(list 'a 'b 'c 'd)` is the item at position 1 of `(list 'b 'c 'd)`, which is the item at position 0 of `(list 'c 'd)`. So `sub1` the index and `rest` the list **together**.

```
;; Produce the item at position n (counting from 0), or empty if there is none.
;; index: Nat (listof Any) -> Any
(define (index n lst)
  (cond [(empty? lst) empty]          ; ran out of list: check this FIRST
        [(zero? n) (first lst)]       ; safe: lst is non-empty here
        [else (index (sub1 n) (rest lst))]))
(check-expect (index 2 (list 'a 'b 'c 'd 'e)) 'c)
(check-expect (index 0 (list 'a)) 'a)
(check-expect (index 100 (list 0 1 2)) empty)
(check-expect (index 0 empty) empty)

;; The first n items (or all of them if there are fewer).
;; first-n: Nat (listof Any) -> (listof Any)
(define (first-n n lst)
  (cond [(or (empty? lst) (zero? n)) empty]
        [else (cons (first lst) (first-n (sub1 n) (rest lst)))]))
(check-expect (first-n 3 (list 1 2 3 4 5 6 7)) (list 1 2 3))
(check-expect (first-n 9 (list 1 2)) (list 1 2))

;; Everything after the first n items.
;; rest-n: Nat (listof Any) -> (listof Any)
(define (rest-n n lst)
  (cond [(or (empty? lst) (zero? n)) lst]
        [else (rest-n (sub1 n) (rest lst))]))
(check-expect (rest-n 3 (list 1 2 3 4 5 6 7)) (list 4 5 6 7))
(check-expect (rest-n 0 (list 1 2)) (list 1 2))
```

TRAP: BASE-CASE ORDER

When more than one argument changes, you must check **every** base case. In `index`, if `(zero? n)` came first, `(index 0 empty)` would call `(first empty)` and crash. Ask: "for each base case, is every operation in its answer safe?"

SHORTCUT

Returning `empty` for "no such item" makes the contract `(anyof X empty)`. The same "failure value" idea returns in dictionary lookup (L12).

L09 · 2

The final Rules for Recursion

RULES FOR RECURSION, FINAL VERSION

1. Change **at least one** argument closer to termination while recurring. Arguments not involved in recursion should not change.
2. When recurring on a natural number, use `(sub1 n)` and test termination with `zero?`.
3. When recurring on a list, use `(rest lst)` and test termination with `empty?`.

Rules 2 and 3 guarantee termination: every recursive call has some argument that is strictly smaller, and it must eventually hit 0 or `empty`. L10 adds one exception: **accumulators** (arguments not involved in termination) may change.

VERSION	WHAT CHANGED
v1 (L04)	exactly one argument changes, Nat only
v2 (L06)	adds lists: `rest` + `empty?`
final (L09)	at least one argument changes; several may move together
with accumulators (L10)	arguments not driving termination may also change

L09 · 3

Appending one list to another

```
;; appnd: (listof Any) (listof Any) -> (listof Any)
(define (appnd lst0 lst1)
  (cond [(empty? lst0) lst1]                                ; nothing left of lst0
        [else (cons (first lst0) (appnd (rest lst0) lst1))])) ; lst1 never changes
(check-expect (appnd (list 1 2 3) (list 'a 'b)) (list 1 2 3 'a 'b))
(check-expect (appnd empty (list 'a)) (list 'a))
(check-expect (appnd (list 1) empty) (list 1))
(check-expect (appnd empty empty) empty)
```

- Recurse on the **first** list only. Its elements get consed back in front of the second list, which is passed along untouched.
- Linear in the length of the **first** list; the second list's length doesn't matter.
- Built-in `append` is allowed from L09 (treat it as linear). It accepts more than two lists: `(append (list 1) (list 2) (list 3))`.

TRAP

`append` joins **lists**. To add one element x at the end write `(append lst (list x))`, not `(append lst x)`. And `(cons lst1 lst2)` is not append.

L09 · 4

Recursion on two lists: the three cases

CASE	WHAT SHRINKS EACH CALL	EXAMPLE
1 · one list only	only lst0; lst1 is passed along	`appnd`
2 · lockstep	both, every call	`dot-product`, string equality
3 · depends on the data	one, the other, or both, chosen by comparing firsts	`merge`

Lockstep

```
;; dot-product: (listof Num) (listof Num) -> Num
(define (dot-product v1 v2)
  (cond [(empty? v1) 0]
        [(empty? v2) 0]
        [else (+ (* (first v1) (first v2))
                 (dot-product (rest v1) (rest v2)))]))
(check-expect (dot-product (list 1 2 3) (list 4 5 6)) 32)
(check-expect (dot-product empty empty) 0)

;; Pair up corresponding elements.
;; zip: (listof Any) (listof Any) -> (listof (list Any Any))
(define (zip lst1 lst2)
  (cond [(or (empty? lst1) (empty? lst2)) empty]
        [else (cons (list (first lst1) (first lst2))
                    (zip (rest lst1) (rest lst2)))]))
(check-expect (zip (list 1 2 3) (list 'a 'b)) (list (list 1 'a) (list 2 'b)))
```

Merging two sorted lists

```
;; merge: (listof Num) (listof Num) -> (listof Num)
;; Requires: both lists sorted in ascending order
(define (merge lst1 lst2)
  (cond [(empty? lst1) lst2]
        [(empty? lst2) lst1]
        [(< (first lst1) (first lst2))
         (cons (first lst1) (merge (rest lst1) lst2))]     ; only lst1 shrinks
        [(> (first lst1) (first lst2))
         (cons (first lst2) (merge lst1 (rest lst2)))]     ; only lst2 shrinks
        [else (cons (first lst1)                           ; equal: take both
                    (cons (first lst2) (merge (rest lst1) (rest lst2))))]))
(check-expect (merge (list 2 4 6 8) (list 1 3 5 7)) (list 1 2 3 4 5 6 7 8))
(check-expect (merge (list 1 2 2 3 3) (list 1 2 3 3)) (list 1 1 2 2 2 3 3 3 3))
(check-expect (merge empty (list 1)) (list 1))
```

STEPS: BASE CASES FOR TWO LISTS

1. List the four combinations: both empty, only the first empty, only the second empty, neither empty.
2. Decide each answer. Often "first empty" covers "both empty" too (as in `merge`, where it produces lst2).
3. Decide what unequal lengths mean (stop? treat missing as 0? error via Requires?) and write a test for it.

;; L10

Accumulative recursion

An extra parameter that carries information down the recursion instead of combining results on the way back up. This is the end of the "fundamentals" half of the course.

YOU SHOULD KNOW

- Accumulators and accumulative recursion
- Reversing a list with an accumulator
- Counting up and down with limits
- How the standard list functions work on lists in lists

L10 · 1

Accumulators and accumulative recursion

An **accumulator** is a parameter that keeps track of partial results or context as you go down the list. It isn't what makes the recursion stop (the list or Nat still does that), so it is allowed to change on every call. A **wrapper** function gives it its starting value and hides it from the user.

STEPS: WRITING AN ACCUMULATOR FUNCTION

1. Decide what you need to remember about the part of the list you've already passed: the answer so far, a running total, the current position, the previous element, the largest so far.
2. Add that parameter to a helper (name it like `fn/acc`) and pick its starting value: `empty` for a list, `0` for a sum or position, `1` for a product, the first element for a max.
3. In the recursive call, recur on `(rest lst)` and pass an **updated** accumulator.
4. In the base case, usually produce the accumulator (or something built from it).
5. Write the wrapper with the original contract: `(define (fn lst) (fn/acc lst start))`.

Two styles to recognise:

- **The accumulator holds the answer** (reverse, sum): the base case returns it.
- **The accumulator holds context** (position, running total, previous item): you still build the answer with `cons` in the normal way, and the base case returns `empty`.

```
;; Style 1: the accumulator IS the answer so far.
;; sum/acc: (listof Num) Num -> Num
(define (sum/acc lon total)
  (cond [(empty? lon) total]
        [else (sum/acc (rest lon) (+ total (first lon)))]))
;; sum-all: (listof Num) -> Num
(define (sum-all lon) (sum/acc lon 0))
(check-expect (sum-all (list 1 2 3 4)) 10)
```

```
(check-expect (sum-all empty) 0)

;; Style 2: the accumulator is context (a running total); output built by cons.
;; running/acc: (listof Num) Num -> (listof Num)
(define (running/acc lon total)
  (cond [(empty? lon) empty]
        [else (cons (+ total (first lon))
                    (running/acc (rest lon) (+ total (first lon))))]))
;; running-sums: (listof Num) -> (listof Num)
(define (running-sums lon) (running/acc lon 0))
(check-expect (running-sums (list 1 2 3 4)) (list 1 3 6 10))

;; Style 2 again: the accumulator is the current position.
;; positions/acc: Sym (listof Sym) Nat -> (listof Nat)
(define (positions/acc target los pos)
  (cond [(empty? los) empty]
        [(symbol=? target (first los))
         (cons pos (positions/acc target (rest los) (add1 pos)))]
        [else (positions/acc target (rest los) (add1 pos))]))
;; positions-of: Sym (listof Sym) -> (listof Nat)
(define (positions-of target los) (positions/acc target los 0))
(check-expect (positions-of 'a (list 'a 'b 'a 'c)) (list 0 2))
```

TRAP

- Building a list by consing onto an accumulator produces it **reversed**. If you need the original order, use plain recursion, or reverse once in the wrapper.
- Wrong starting value in the wrapper (0 for a product, for example).
- Don't reach for an accumulator for a plain fold, map or filter. Use one when you need information about elements you've already passed.

L10 · 2

Reversing a list with an accumulator

Move elements one at a time from the front of the list onto the front of the accumulator.
The accumulator ends up reversed.

```
;; rev/acc: (listof Any) (listof Any) -> (listof Any)
(define (rev/acc lst acc)
  (cond [(empty? lst) acc]
        [else (rev/acc (rest lst) (cons (first lst) acc))]))
;; rev: (listof Any) -> (listof Any)
(define (rev lst) (rev/acc lst empty))
(check-expect (rev (list 1 2 3 4 5)) (list 5 4 3 2 1))
(check-expect (rev empty) empty)
```

```
(rev/acc (list 1 2 3) empty)
→ (rev/acc (list 2 3) (list 1))
→ (rev/acc (list 3)   (list 2 1))
→ (rev/acc empty      (list 3 2 1))
→ (list 3 2 1)
```

- Linear: 7n + 5 steps, versus 3.5n² + 6.5n + 4 for `rev-slow` with `attach`.
- Built-in `reverse` is allowed from L10 (treat it as linear). Use it once in a wrapper if an answer comes out backwards; never call it inside every recursive step.

L10 · 3

Counting up and down with limits

Counting **down** is natural with `sub1`. Counting **up** while still following the Rules needs one of three tricks: arithmetic with a limit, an accumulator that counts up, or reversing.

```
;; count-down: Nat -> (listof Nat)      n, n-1, ..., 0   (n+1 items)
(define (count-down n)
  (cond [(zero? n) (list 0)]
        [else (cons n (count-down (sub1 n)))]))
(check-expect (count-down 3) (list 3 2 1 0))

;; Trick 1, arithmetic: produce limit-n, ..., limit-1.
;; count-up-to: Nat Nat -> (listof Nat)
(define (count-up-to limit n)
  (cond [(zero? n) empty]
        [else (cons (- limit n) (count-up-to limit (sub1 n)))]))
(check-expect (count-up-to 105 5) (list 100 101 102 103 104))

;; Trick 2, accumulator: from counts up while n counts down.
;; count-up-from: Nat Nat -> (listof Nat)    n values starting at from
(define (count-up-from from n)
  (cond [(zero? n) empty]
        [else (cons from (count-up-from (add1 from) (sub1 n)))]))
(check-expect (count-up-from 100 5) (list 100 101 102 103 104))

;; count-up: Nat -> (listof Nat)          0, 1, ..., n-1   (n items)
(define (count-up n) (count-up-from 0 n))
(check-expect (count-up 4) (list 0 1 2 3))
(check-expect (count-up 0) empty)

;; Inclusive range a..b, then map the values if needed.
;; range-incl: Nat Nat -> (listof Nat)
;; Requires: a <= b
(define (range-incl a b) (count-up-from a (add1 (- b a))))
(check-expect (range-incl 3 6) (list 3 4 5 6))
(check-expect (add1-list (count-up 3)) (list 1 2 3))
```

TRAP: OFF BY ONE

Compare base cases. `count-down` returns `(list 0)`, so it includes 0 and has n+1 items. `countdown` from L08 returns `empty`, so it stops at 1. `count-up n` stops at n−1. Always check with n = 0 and n = 1.

L10 · 4

Standard list functions on lists in lists

A list is a value, so a list can be an element of another list. `cons first rest empty? length append reverse` all work exactly as before, on the **outer** list only.

```
(define colours
  (list (list 'glint  (list 245 196 71))
        (list 'ember  (list 218 82 48))
        (list 'citron (list 211 224 84))))
(check-expect (first (first (rest colours))) 'ember)
(check-expect (first (rest (first colours))) (list 245 196 71))
(check-expect (first (first (rest (first (rest colours))))) 218)
(check-expect (cons 0 (first (rest (first colours)))) (list 0 245 196 71))
(check-expect (length colours) 3)                       ; top level only
(check-expect (reverse (list (list 1 2) (list 3))) (list (list 3) (list 1 2)))
```

STEPS: EVALUATING A NESTED FIRST/REST CHAIN

1. Start from the innermost call on the named list and write down what it produces.
2. Apply the next function outward to *that* value. `(rest colours)` → list of the last two colours; `(first …)` → `(list 'ember (list 218 82 48))`; `(rest …)` → `(list (list 218 82 48))`; `(first …)` → `(list 218 82 48)`; `(first …)` → `218`.
3. Keep track of "list of X" vs "an X" at every stage. That is where the mistakes happen.

;; L11

Lists of lists

Nothing new in the language: the same techniques applied to lists whose elements are lists. Polygons, tables and subsets.

YOU SHOULD KNOW

- How the techniques from earlier lectures generalize to lists of lists

---

L11 · 1

A list of points: polygon perimeter

```
;; d: Point Point -> Num      distance between two points (Point is (list Num Num))
(define (d p0 p1)
  (sqrt (+ (sqr (- (second p1) (second p0)))
           (sqr (- (first p1) (first p0))))))
(check-within (d (list 1 5) (list 4 1)) 5 0.00001)

;; Sum the segments between consecutive points (does NOT close the shape).
;; add-segments: (listof Point) -> Num
(define (add-segments lop)
  (cond [(or (empty? lop) (empty? (rest lop))) 0]       ; adjacent-pair pattern
        [else (+ (d (first lop) (second lop)) (add-segments (rest lop)))]))

;; Version 1: open path + closing segment (walks the list twice).
;; perimeter: (listof Point) -> Num
(define (perimeter poly)
  (+ (d (first poly) (last poly)) (add-segments poly)))

;; Version 2: carry the first point along so the last segment can close the loop.
;; add-segments/wrap: Point (listof Point) -> Num
(define (add-segments/wrap p0 lop)
  (cond [(empty? lop) 0]
        [(empty? (rest lop)) (d p0 (first lop))]        ; last point back to the start
        [else (+ (d (first lop) (second lop)) (add-segments/wrap p0 (rest lop)))]))
;; perimeter/v2: (listof Point) -> Num
(define (perimeter/v2 poly) (add-segments/wrap (first poly) poly))

(define square4 (list (list 1 1) (list -1 1) (list -1 -1) (list 1 -1)))
(check-within (add-segments square4) 6 0.00001)
(check-within (perimeter square4) 8 0.00001)
(check-within (perimeter/v2 square4) 8 0.00001)
```

SHORTCUT: CARRY THE ORIGINAL

For wrap-around problems (closing a polygon, comparing the last element to the first), pass the original first element along as an extra parameter that never changes.

L11 · 2

Tables: one function per level

A table is a `(listof (listof X))`: a list of rows. The outer function recurses over rows and hands each row to a helper that recurses over that row's elements.

```
;; sum-row: (listof Num) -> Num
(define (sum-row row)
  (cond [(empty? row) 0]
        [else (+ (first row) (sum-row (rest row)))]))
;; sum-table: (listof (listof Num)) -> Num          fold over rows
(define (sum-table t)
  (cond [(empty? t) 0]
        [else (+ (sum-row (first t)) (sum-table (rest t)))]))
;; row-sums: (listof (listof Num)) -> (listof Num)  map over rows
(define (row-sums t)
  (cond [(empty? t) empty]
        [else (cons (sum-row (first t)) (row-sums (rest t)))]))
;; flatten: (listof (listof Any)) -> (listof Any)
(define (flatten t)
  (cond [(empty? t) empty]
        [else (append (first t) (flatten (rest t)))]))
(define tbl (list (list 1 2 3) (list 4 5) empty))
(check-expect (sum-table tbl) 15)
(check-expect (row-sums tbl) (list 6 9 0))
(check-expect (flatten tbl) (list 1 2 3 4 5))
```

Generating a table

```
;; One row: n*m, n*(m+1), ..., n*(m+i)   (i+1 entries)
;; times/row: Nat Nat Nat -> (listof Nat)
(define (times/row n m i)
  (cond [(zero? i) (list (* n m))]
        [else (cons (* n m) (times/row n (add1 m) (sub1 i)))]))  ; m counts up, i controls
;; Rows for n, n+1, ..., n+i, each with columns 0..m
;; times/rc: Nat Nat Nat -> (listof (listof Nat))
(define (times/rc n m i)
  (cond [(zero? i) (list (times/row n 0 m))]
        [else (cons (times/row n 0 m) (times/rc (add1 n) m (sub1 i)))]))
;; times: Nat Nat -> (listof (listof Nat))
(define (times n m) (times/rc 0 m n))
(check-expect (times/row 3 0 5) (list 0 3 6 9 12 15))
(check-expect (times 3 5) (list (list 0 0 0 0 0 0)
                                (list 0 1 2 3 4 5)
                                (list 0 2 4 6 8 10)
                                (list 0 3 6 9 12 15)))
```

TRAP: OFF BY ONE

`(times 3 5)` has **4 rows and 6 columns** (0..3 by 0..5) because each base case at zero still produces an entry. Read base cases carefully when counting outputs.

Indexing into a table

```
;; index-table: Nat Nat (listof (listof Any)) -> Any
(define (index-table n m table) (index m (index n table)))   ; row n, then column m
(check-expect (index-table 2 3 (list (list 1 2 3 4 5)
                                     (list 'a 'b 'c 'd 'e 'f)
                                     (list 'u 'v 'w 'x 'y 'z)))
              'x)
(check-expect (index-table 100 100 empty) empty)
```

Composition works because `index` produces `empty` when out of range, and `empty` is also a (list of) list, so the outer `index` just produces `empty` again.

L11 · 3

Subsets: solve the rest, then expand with the first

```
;; A Set is a (listof Sym) with no duplicates.
;; For each subset produce two: one with sym, one without. Plus (list sym) alone.
;; expand: Sym (listof Set) -> (listof Set)
(define (expand sym subs)
  (cond [(empty? subs) (list (list sym))]
        [else (cons (cons sym (first subs))
                    (cons (first subs) (expand sym (rest subs))))]))
;; All non-empty subsets.
;; subsets: Set -> (listof Set)
(define (subsets s)
  (cond [(empty? s) empty]
        [else (expand (first s) (subsets (rest s)))]))
(check-expect (subsets (list 'a 'b 'c))
              (list (list 'a 'b 'c) (list 'b 'c) (list 'a 'c) (list 'c)
                    (list 'a 'b) (list 'b) (list 'a)))
(check-expect (subsets empty) empty)
```

- A set of n symbols has $2^n - 1$ non-empty subsets, so the output itself grows exponentially. That's the size of the answer, not a blow-up bug: `subsets` makes one recursive call.
- Same shape as insertion sort: `f(lst) = helper(first, f(rest))`, where the helper walks the result of the recursion.

;; L12

Strings and dictionaries

Characters and Unicode, strings processed as lists of characters, lexicographic order, and dictionaries implemented as association lists.

YOU SHOULD KNOW

- How Unicode makes information interchange possible
- How characters relate to strings
- Converting between strings and lists of characters
- Processing strings as lists of characters
- The dictionary as an abstract data type
- Implementing a dictionary with an association list

L12 · 1

Unicode and information interchange

- Computers store everything as numbers. To exchange text, everyone must agree which number means which character.
- **Unicode** assigns a unique number, a **code point**, to almost every character in almost every written language. Written in base 16 with a `U+` prefix: U+0061 = 97 = "a".
- Code points below 128 come from the older **ASCII** standard: English letters, digits, punctuation.
- A **character** is the abstract thing (a code point); a **glyph** is how a font draws it. Same code point in different fonts = same character. Latin "A" (65), Greek "A" (913) and Cyrillic "A" (1040) look identical but are different characters.
- "a" and "A" are different characters. For CS135, two characters are the same exactly when their code points are the same.

CHARACTER	CODE POINT
space	32
`#\0` … `#\9`	48 … 57
`#\A` … `#\Z`	65 … 90
`#\a` … `#\z`	97 … 122

SHORTCUT: ORDERING

Space < digits < UPPERCASE < lowercase. So `(char<? #\a #\B)` is `false`, and `(string<? "Zebra" "apple")` is `true`.

L12 · 2

Characters and strings

- A character literal is `#\` followed by the character: `#\a`, `#\Z`, `#\7`, `#\space`. Type: **Char**.
- A string is a sequence of zero or more characters in double quotes: `"Hello"`, `""`. Type: **Str**.
- Escapes: `\"` puts a double quote inside a string; `\\` puts one backslash.
- Allowed: `char? char=? char<?` (and `char<=? char>? char>=?`), `string? string=? string<?` (and `string<=? string>? string>=?`), `string->list list->string`. **Nothing else** without permission: no `string-length`, `substring`, `char-upcase`.

```
;; lower-case?: Char -> Bool      test a range with comparisons
(define (lower-case? c) (and (char<=? #\a c) (char<=? c #\z)))
(check-expect (lower-case? #\q) true)
(check-expect (lower-case? #\Q) false)
(check-expect (char<? #\a #\B) false)
(check-expect (char<? #\Z #\a) true)
```

SHORTCUT

Length of a string without `string-length`: `(length (string->list s))`.

L12 · 3

Converting between strings and lists of characters

```
;; string->list: Str -> (listof Char)
;; list->string: (listof Char) -> Str
(check-expect (string->list "a b") (list #\a #\space #\b))
(check-expect (list->string (list #\h #\i)) "hi")
(check-expect (string->list "") empty)
(check-expect (list->string empty) "")
```

TRAP

Spaces become `#\space` elements and count toward the length. A function whose contract says `-> Str` must end with `list->string` ; returning the list of characters fails every test.

L12 · 4

Processing strings as lists of characters

STEPS: THE STRING PATTERN

1. **Wrapper** (contract uses Str): convert each string with `string->list` .
2. **Helper** (contract uses `(listof Char)`): an ordinary list function: fold, filter, map, lockstep.
3. If the answer is a string, convert back with `list->string` in the wrapper.

```
;; Fold: count occurrences of a character.
;; count-char/list: Char (listof Char) -> Nat
(define (count-char/list c loc)
  (cond [(empty? loc) 0]
        [(char=? c (first loc)) (add1 (count-char/list c (rest loc)))]
        [else (count-char/list c (rest loc))]))
;; count-char: Char Str -> Nat
(define (count-char c s) (count-char/list c (string->list s)))
(check-expect (count-char #\l "hello world") 3)

;; Filter: remove spaces, producing a Str.
;; no-spaces/list: (listof Char) -> (listof Char)
(define (no-spaces/list loc)
  (cond [(empty? loc) empty]
        [(char=? #\space (first loc)) (no-spaces/list (rest loc))]
        [else (cons (first loc) (no-spaces/list (rest loc)))]))
;; no-spaces: Str -> Str
(define (no-spaces s) (list->string (no-spaces/list (string->list s))))
(check-expect (no-spaces "a b  c") "abc")

;; Lockstep: string equality written by hand.
;; char-list=?: (listof Char) (listof Char) -> Bool
(define (char-list=? loc0 loc1)
  (cond [(empty? loc0) (empty? loc1)]      ; equal only if both ran out together
        [(empty? loc1) false]
        [else (and (char=? (first loc0) (first loc1))
                   (char-list=? (rest loc0) (rest loc1)))]))
;; str=?: Str Str -> Bool
(define (str=? s0 s1) (char-list=? (string->list s0) (string->list s1)))
(check-expect (str=? "hello" "hello") true)
(check-expect (str=? "hello" "help") false)
```

```
(check-expect (str=? "" "") true)
(check-expect (str=? "ab" "abc") false)
```

L12 · 5

Lexicographic order

1. Compare character by character from the left.
2. At the first position where they differ, the smaller code point comes first.
3. If one string runs out first (it's a prefix of the other), the **shorter** one comes first. `""` comes before every non-empty string.

```
;; char-list<?: (listof Char) (listof Char) -> Bool
(define (char-list<? loc0 loc1)
  (cond [(empty? loc0) (not (empty? loc1))]     ; prefix (or equal) case
        [(empty? loc1) false]
        [(char=? (first loc0) (first loc1)) (char-list<? (rest loc0) (rest loc1))]
        [else (char<? (first loc0) (first loc1))]))   ; first difference decides
;; str<?: Str Str -> Bool
(define (str<? s0 s1) (char-list<? (string->list s0) (string->list s1)))
(check-expect (str<? "planet" "plants") true)
(check-expect (str<? "plan" "plants") true)
(check-expect (str<? "" "a") true)
(check-expect (str<? "abc" "abc") false)
(check-expect (str<? "Zebra" "apple") true)
(check-expect (string<? "10" "9") true)
```

TRAP

Lexicographic is not numeric: `"10"` < `"9"` because `#\1` < `#\9`. And capitals sort before every lowercase letter.

L12 · 6

The dictionary abstract data type

- A **dictionary** stores **key-value pairs**. Keys are **unique**; values may repeat (two students can have the same mark).
- Operations: **lookup** (also find/search): key → its value; **add** (insert): store a pair; **delete** (remove): drop a key and its value.
- It's an **abstract** data type: it says what the operations do, not how they're stored. An association list is one implementation.
- Examples: word → definition, name → phone number, user ID → seat, stock symbol → price.

L12 · 7

Association lists

```
;; An association list (AL) is a (listof (list Str Num))
;; Requires: each key (Str) is unique
(define marks (list (list "asmith" 87) (list "bjones" 64) (list "cwang" 87)))

;; lookup: Str AL -> (anyof Num empty)
(define (lookup key al)
  (cond [(empty? al) empty]                                  ; not found
        [(string=? key (first (first al))) (second (first al))] ; key of 1st pair matches
        [else (lookup key (rest al))]))
(check-expect (lookup "bjones" marks) 64)
(check-expect (lookup "nobody" marks) empty)

;; delete: Str AL -> AL
(define (delete key al)
  (cond [(empty? al) empty]
        [(string=? key (first (first al))) (rest al)]          ; keys unique: stop here
        [else (cons (first al) (delete key (rest al)))]))
(check-expect (delete "bjones" marks) (list (list "asmith" 87) (list "cwang" 87)))
(check-expect (delete "x" empty) empty)

;; add: Str Num AL -> AL       new or replaced pair goes on the FRONT
(define (add key value al) (cons (list key value) (delete key al)))
(check-expect (add "dlee" 48 marks)
              (list (list "dlee" 48) (list "asmith" 87) (list "bjones" 64) (list "cwang" 87)))
(check-expect (add "cwang" 100 marks)
              (list (list "cwang" 100) (list "asmith" 87) (list "bjones" 64)))
```

- **Why `empty` for a failed lookup?** Any number could be a real value, so failure must be something that can't be confused with one. Contract: `(anyof Num empty)`.
- **Why delete stops at the first match:** keys are unique, so there's nothing left to remove. Return `(rest al)`.
- **Why add deletes first:** to keep keys unique when the key already exists. This makes add double as "update".

TRAP

- The key is `(first (first al))` and the value is `(second (first al))`. Returning `(first al)` returns the whole pair.
- Use the equality for the key type: `string=?` for Str, `symbol=?` for Sym, `=` for Num.
- `add` puts the pair at the **front**; expected lists in `check-expect` must match that order exactly.

;; how to write it down

Design recipe & style

How to lay out a function so it earns the purpose, contract and test-coverage marks. Everything here follows the formats the slides use in L02–L12. The official CS135 style guide (linked in L03) needs a UW login, so it isn't summarised here. Read it once, and where it differs from this page, follow it.

ON THIS PAGE

- A complete answer, top to bottom
- Writing the purpose
- Writing the contract and Requires
- Picking the template
- How to format a test
- Which tests to write, and how many
- Data definitions, helpers and wrappers
- Layout: comments, indentation, names, constants

Recipe · 1

A complete answer, top to bottom

This is what one assignment file looks like with every piece in place. Copy the shape.

```
;;
;; *************************************************
;; Your Name (12345678)
;; CS 135 Fall 2026
;; Assignment 05, Problem 2
;; *************************************************
;;

;;
;; Question 2. Part a.
;;

(define pass-mark 50)   ; lowest passing mark

;; Count the marks in marks that are at least pass-mark.
;; count-passing: (listof Nat) -> Nat
(define (count-passing marks)
  (cond [(empty? marks) 0]
        [(>= (first marks) pass-mark)
         (add1 (count-passing (rest marks)))]
        [else (count-passing (rest marks))]))
```

```
(check-expect (count-passing empty) 0)
(check-expect (count-passing (list 50)) 1)    ; exactly at the threshold
(check-expect (count-passing (list 49)) 0)    ; just below it
(check-expect (count-passing (list 90 12 50 49 100)) 3)

;;
;; Question 2. Part b.
;;

;; Compute the length of the hypotenuse of a right triangle with legs a and b.
;; hypotenuse: Num Num -> Num
;; Requires: a > 0 and b > 0
(define (hypotenuse a b)
  (sqrt (+ (sqr a) (sqr b))))

(check-expect (hypotenuse 3 4) 5)                 ; perfect square: result is exact
(check-within (hypotenuse 1 1) 1.41421 0.00001)  ; inexact result: check-within
```

PIECE	WHERE AND HOW
File header	Top of every file: name, student ID, term, assignment and problem. Any clear format is accepted. This is the one from the slides.
Part label	`;; Question 2. Part a.` before each part, in the order the assignment lists them.
Constants	Named values instead of bare numbers, above the function that uses them.
Purpose	One `;;` sentence saying what the function produces from what it consumes.
Contract	`;; name: Type Type -> Type`, on the line under the purpose.
Requires	Only if needed, on the line under the contract.
Definition	Header and body, indented by DrRacket.
Tests	`check-expect` / `check-within` after the function, enough for full coverage.

SHORTCUT: THE ORDER YOU WRITE IN ISN'T THE ORDER ON THE PAGE

You *write* purpose → contract and header → examples → body → extra tests. On the page the examples just sit with the other tests below the definition, as the slides show. Racket runs every test after all the definitions, so where a test sits doesn't change whether it runs.

Recipe · 2

Writing the purpose

The purpose says what the function does: what it produces and what it consumes, wherever the contract alone doesn't make that clear. It is graded on its own (5% by

default).

STEPS

1. Start with a verb that names the output: *Compute*, *Count*, *Produce*, *Determine if*, *Insert*, *Filter*, *Convert*. For a predicate, a yes/no question works too: `;; Are two strings equal?`
2. Name the inputs, using parameter names where that's clearer: `;; Count how many times sym appears in lst.`
3. Add anything the types can't say: "of a **non-empty** list", "into lon, which is **sorted**", "produce `empty` if i is out of range".
4. Keep it to one or two lines. Say **what** comes out, not **how**: never "uses recursion to ...".

WEAK	BETTER
`;; recursion on a list`	`;; Count the apples in a grocery list.`
`;; adds things up`	`;; Compute the sum of the numbers in lon.`
`;; removes apples`	`;; Produce los with every 'apple removed.`
`;; last element`	`;; Produce the last element of a non-empty list.`
`;; checks the card`	`;; Determine if card is a face card (jack, queen or king).`

ON THE EXAM

If the function only works on a non-empty list (largest, last, smallest), L07 says the purpose must say so, and the contract needs a Requires line as well. If an exam question already gives you the purpose or header, copy its names exactly.

Recipe · 3

Writing the contract and Requires

```
;; name: Type1 Type2 -> ResultType
;; Requires: a condition the types can't express
```

- Function name exactly as in the `define`, then a colon.
- **One type per parameter, in the header's order.** Two parameters → two types before `->`.
- Exactly one result type after `->`. If the result can be one of several kinds, wrap them in `(anyof …)`.
- Course type names, capitalised: `Nat Int Rat Num Bool Sym Char Str Any`. Plus `(listof X)`, `(anyof …)`, `(list A B)` for a fixed-length list, and names from your data definitions (`Point`, `Card`, `AL`).

IF THE VALUE IS …	USE
a count, length, index or position	`Nat`
a whole number that can be negative	`Int`
the result of `/`, `sqrt`, or arithmetic on Num inputs	`Num`
true or false	`Bool` (and the name ends in `?`)
one of a fixed set of symbols	`Sym`, `(anyof 'jacket 'sweater 'shirt)`, or a named type such as `Outerwear`
a list whose element type you never use	`(listof Any)`
an answer, or `empty` when there is none	`(anyof Num empty)`
a pair like `(list "asmith" 87)`	`(list Str Num)`
a list of those pairs	`(listof (list Str Num))`
a list of lists of numbers	`(listof (listof Num))`

Requires

Requires is for limits the types can't express. Callers promise to respect it, so you don't handle or test inputs that break it.

```
;; Requires: lst is non-empty
;; Requires: lon is sorted in ascending order
;; Requires: each key (Str) is unique
;; Requires: lst0 and lst1 have the same length
;; Requires: n >= 1
;; Requires: temperature >= -273.15
```

TRAP: CONTRACT MISTAKES THAT COST MARKS

- An average or any division with `-> Nat`: `(/ 7 2)` is `7/2`, so it's `-> Num`.
- `-> Nat` when the answer can be negative (a difference, a minimum of Ints).
- Fewer types than parameters, most often a helper's contract that leaves out the accumulator.
- `Boolean`, `Number`, `Symbol`, `String`, `List`: the course names are `Bool Num Sym Str (listof X)`.
- No Requires on a function that breaks on `empty` or on unsorted input.

Recipe · 4

Picking the template (step 4)

The input's shape picks the template. Copy the template, rename it to your function and parameters, then fill the blanks with the four questions from L06 · 1.

THE INPUT IS …	TEMPLATE	BASE CASE
a Nat you count down	Nat (L04)	`(zero? n)`
a list	list (L06)	`(empty? lst)`
a list that must be non-empty (largest, last)	non-empty list (L07)	`(empty? (rest lst))`
a Nat and a list together (index, first-n)	Nat + list (L09)	`(zero? n)` and `(empty? lst)`
two lists, only one taken apart (append)	one list (L09)	`(empty? lst0)`
two lists walked together (dot product)	lockstep (L09)	both empty
two sorted lists (merge)	merge (L09)	either one empty
counting up, a running value, or an answer that comes out backwards	accumulator + wrapper (L10)	produce the accumulator
a string	wrapper with `string->list` (L12)	`(empty? loc)`

SHORTCUT

Once the template is chosen, the Pattern library usually has the filled-in shape for the question's wording.

Recipe · 5

How to format a test

```
(check-expect  (function-call …)  expected-value)
(check-within  (function-call …)  expected-value  delta)
```

- **The call you're testing goes first, the answer second.**
- The expected value is a plain value you worked out **by hand**: `6`, `true`, `'jacket`, `(list 1 2 3)`, `empty`, `"abc"`, `#\a`. Never compute it by calling your own function or copying its code: that test passes even when the function is wrong.

- Use `check-within` whenever the result is inexact: `sqrt` of a non-square, `sin`, `pi`, anything shown with `#i`. In our language, `check-expect` on an inexact result is an **error**. The question usually gives the delta.
- Exact fractions and decimals are fine in `check-expect`: decimals like `3.5` are exact in our language.
- With list abbreviations, write expected lists with `list`. `(list 1 2)` and `(cons 1 (cons 2 empty))` are the same value.

```
;; Exact results: check-expect, even for fractions and decimals.
(check-expect (/ 7 2) 7/2)
(check-expect (/ 7 2) 3.5)

;; Long tests: put the call and the expected value on separate lines.
(check-expect
 (insert 3 (list 1 2 4 5))
 (list 1 2 3 4 5))

;; Big inputs: name them once and reuse them (L07 does this).
(define class-marks (list 88 49 50 73 12 100 51))
(check-expect (count-passing class-marks) 5)

;; Inexact results: check-within, with the delta the question gives.
(check-within (hypotenuse 1 2) 2.23607 0.00001)
```

TRAP: TESTS THAT LOOK FINE BUT AREN'T

```
(check-expect (count-passing (list 60 70))
              (count-passing (list 60 70)))  ; proves nothing: passes even if it's wrong
(check-expect (hypotenuse 1 1) 1.41421)       ; error: inexact result needs check-within
(check-expect (last empty) empty)             ; breaks "Requires: lst is non-empty"
(check-expect (factorial -1) 1)               ; breaks the contract: -1 isn't a Nat
(check-expect (factorial 3) (* 3 2 1))        ; works, but just write 6
```

Recipe · 6

Which tests to write, and how many

The minimum for assignments is **full coverage**: after running, no code is highlighted. In the slides' words, it's hard to test too much. Coverage is 10% of the marks by default, and hidden tests decide the other 80%, so your own tests are how you find bugs first.

STEPS: BUILD THE TEST LIST

1. Turn every example in the question into a `check-expect` first.
2. Test the base case: the smallest legal input (`0` , `empty` , `""` , or a one-element list if the function requires non-empty).
3. For each `cond` question except `else` : one test where it's true and one where it's false (with the earlier questions false).
4. For each threshold, test at it and just on each side. Thresholds 8 and 16 → 7, 8, 15, 16.
5. For a search, put the answer at the front, in the middle and at the end, and once not there at all.
6. Give every helper and wrapper its own tests. Test an accumulator helper with an accumulator that isn't the starting value.
7. Run in DrRacket. Highlighted code never ran: add a test that reaches it.

INPUT	TESTS TO INCLUDE
Nat	0, 1, a bigger n
Num with thresholds	below, at and above each one; a negative; 0
a fixed set of symbols	each symbol at least once
list	empty; one element; several; match first / middle / last; all match; none match; duplicates
non-empty list	one element; answer first; answer last
sorted list (insert)	into empty; at the front; in the middle; at the end; equal to an existing value
two lists	both empty; each one empty; same length; different lengths
Bool result	at least one `true` and one `false`
string	`""` ; one character; upper vs lower case; spaces or punctuation
dictionary (AL)	empty AL; key first / middle / last; key missing
list of lists	empty outer list; an empty inner list; inner lists of different lengths

SHORTCUT: DON'T TEST OUTSIDE THE CONTRACT

If the contract says Nat, don't test −1 or 2.5. If it requires a non-empty list, don't test `empty` . L04 says this about factorial: since it consumes a Nat, negative and inexact inputs aren't your concern.

ON THE EXAM

When a question asks for a fixed number of tests, make each one take a **different path** through the `cond`. Three tests that all take the same branch show very little.

Recipe · 7

Data definitions, helpers and wrappers

Data definitions

```
;; A Point is an (x, y) point in the Cartesian coordinate system
;; A Point is a (cons Num (cons Num empty))

;; An Outerwear is (anyof 'jacket 'sweater 'shirt)

;; A (listof Food) is one of:
;; * empty
;; * (cons Food (listof Food))

;; An association list (AL) is a (listof (list Str Num))
;; Requires: each key (Str) is unique
```

- A capitalised name and "A … is …". An optional first line says what the data means.
- A recursive definition says "is one of:" with one bullet per case. Its template gets one `cond` clause per bullet.
- Constructors (`mk-point`) and accessors (`get-x`) are functions, so they get a purpose and a contract too.
- Once a type is defined, use its name in contracts: `distance-to-origin: Point -> Num`.

Helpers and wrappers

```
;; Compute the sum of the numbers in lon.
;; total: (listof Num) -> Num
(define (total lon) (total/acc lon 0))

;; Add the numbers in lon to sum-so-far.
;; total/acc: (listof Num) Num -> Num
(define (total/acc lon sum-so-far)
  (cond [(empty? lon) sum-so-far]
        [else (total/acc (rest lon) (+ (first lon) sum-so-far))]))

(check-expect (total empty) 0)
(check-expect (total (list 1 2 3)) 6)
(check-expect (total/acc empty 5) 5)          ; helper tested on its own,
(check-expect (total/acc (list 1 2) 10) 13)   ; with a non-zero accumulator
```

- **Helpers you invent:** from L08 on, a missing purpose or contract doesn't cost marks, but the code still needs full coverage. Write both anyway: it takes seconds, and the contract catches type mistakes.
- **A helper the question names** gets the full recipe.
- **Accumulator helpers:** name them `fn/acc` (the slides use `rev/accumulate`). The purpose says what the accumulator holds, and the contract includes its type.
- **The wrapper** has the contract the question asked for. It only calls the helper with the starting accumulator.
- Functions can call helpers defined above or below them. Keep each helper next to the function that uses it.

Recipe · 8

Layout: comments, indentation, names, constants

- **Comments:** `;;` for a whole line, a single `;` after code on the same line.
- **Line length:** at most 102 characters, DrRacket's default.
- **Brackets:** square around each `cond` pair, round everywhere else.
- **One `cond` pair per line.** If an answer is long, put it on the next line, indented under its question (like `count-passing` above).
- **Closing parentheses** stack at the end of the last line, not on lines of their own (the usual Racket style).
- **Let DrRacket indent:** Racket → Reindent All (Cmd-I on a Mac, Ctrl-I on Windows) fixes the indentation of the whole file.
- **Function names:** lowercase words joined by hyphens (`count-apples`). Predicates end in `?`. Constructors start with `mk-` and accessors with `get-`. The slides prefer a long, clear name like `distance-to-origin` to a cryptic one like `d0`.
- **Parameter names:** `lst`, `lon` (list of numbers), `los` (symbols), `loc` (characters), `n`, `sym`. Never `list`, which hides the built-in.
- **No built-in names** for your own functions (`length`, `append`, `remove-all` ...). Racket refuses with "this name was defined in the language", and then **nothing in the file runs**.
- **Constants** instead of bare numbers: `(define cold 8)`, as in the L03 submission example.
- **Commenting out:** Racket → Comment Out with Semicolons, or `#| … |#`. **Never** Comment Out with a Box: it can't be marked.

TRAP: SAME CODE, UNREADABLE

```
(define (count-passing marks) (cond [(empty? marks) 0] [(>= (first marks) pass-mark) (add1
(count-passing (rest marks)))] [else (count-passing (rest marks))]))
```

It runs, but it breaks the line limit and hides the `cond` structure. Compare it with the version in Recipe · 1.

RECIPE TRAPS

Contract and header disagree

Three parameters but two types, or the types are in a different order from the parameters.

Purpose says how, not what

"Recurses through the list" earns nothing. Say what is produced, from which inputs.

Missing Requires

The function breaks on `empty` or on unsorted input, and the contract doesn't say so.

Only "normal" tests

The base case or the `else` branch is never tested, so the code stays highlighted.

Expected value from your own code

The expected side calls the function being tested, so the test always passes.

`check-expect` on an inexact result

That's an error, not a failed test. Use `check-within`.

Testing outside the contract

`(factorial -1)` or `(last empty)`: behaviour there is undefined, so don't write the test.

Untested helper

Helpers can skip the purpose and contract, but not coverage.

Redefining a built-in

A function named `length` or `remove-all` stops the whole file from running.

;; across the course

Pattern library

Exam questions are mostly the same two dozen shapes in new clothes. Match the wording of the question to a shape, then fill in the blanks. `...` marks the problem-specific parts.

FOLDING: ONE VALUE OUT

Count the elements that ...

"How many ...", "count the ...". Base 0; `add1` on a match, plain recursion otherwise.

```
(cond [(empty? lst) 0]
      [(matches? (first lst)) (add1 (f (rest lst)))]
      [else (f (rest lst))])
```

Sum / product / total of ...

Base is the identity: 0 for `+`, 1 for `*`. To total only some elements, add a clause that skips the others.

```
(cond [(empty? lst) 0]
      [else (+ (value-of (first lst)) (f (rest lst)))])
```

Is every element ...? / Is any element ...?

"All", "every", "only" → base `true`, stop with `false`. "Any", "some", "contains", "exists" → base `false`, stop with `true`.

```
;; all
(cond [(empty? lst) true]
      [(fails? (first lst)) false]
      [else (f (rest lst))])
```

```
;; any
(cond [(empty? lst) false]
      [(passes? (first lst)) true]
      [else (f (rest lst))])
```

Largest / smallest / last (needs a non-empty list)

No sensible answer for `empty` → base on `(empty? (rest lst))`, add "Requires: non-empty". Call the recursion **once**.

```
(cond [(empty? (rest lst)) (first lst)]
      [else (max (first lst) (f (rest lst)))])   ; min for smallest
```

Remove / keep only / filter out ...

Delete an element by **not** consing it; keep it by consing it onto the recursive result.

```
(cond [(empty? lst) empty]
      [(drop? (first lst)) (f (rest lst))]
      [else (cons (first lst) (f (rest lst)))])
```

Remove only the first occurrence / stop after the first match

On the match, produce `(rest lst)` and stop. Also the shape of `insert` and dictionary `delete`.

```
(cond [(empty? lst) empty]
      [(target? (first lst)) (rest lst)]
      [else (cons (first lst) (f (rest lst)))])
```

Transform / convert / add ... to each element

Same length out. Every non-empty branch conses something, possibly something different per branch (replace).

```
(cond [(empty? lst) empty]
      [(special? (first lst)) (cons replacement (f (rest lst)))]
      [else (cons (transform (first lst)) (f (rest lst)))])
```

Remove duplicates

Filter with a membership helper: drop `(first lst)` if it appears again in `(rest lst)`. Keeps the last copy of each value. Quadratic, which is fine.

```
;; num-member?: Num (listof Num) -> Bool
(define (num-member? x lon)
  (cond [(empty? lon) false]
        [(= x (first lon)) true]
        [else (num-member? x (rest lon))]))
;; dedup: (listof Num) -> (listof Num)
(define (dedup lon)
  (cond [(empty? lon) empty]
        [(num-member? (first lon) (rest lon)) (dedup (rest lon))]
        [else (cons (first lon) (dedup (rest lon)))]))
(check-expect (dedup (list 1 2 1 3 2)) (list 1 3 2))
```

Build a list from a number (countdown, range, n copies)

Recurse on the Nat; base `empty` (or `(list 0)` to include 0). Increasing order needs an accumulator counting up (`count-up-from`).

```
(cond [(zero? n) empty]
      [else (cons (item-for n) (f (sub1 n)))])
```

ORDER AND NEIGHBOURS

Is it sorted? / compare each element to the next

Two base cases; look at `(first lst)` and `(second lst)`; recur on `(rest lst)`.

```
(cond [(or (empty? lst) (empty? (rest lst))) …]
      [(bad-pair? (first lst) (second lst)) …]
      [else (f (rest lst))])
```

Insert in order / sort

`insert`: empty → one-element list; belongs here → `(cons n lst)`; else keep first and insert into rest. `sort` = insert first into sorted rest.

TWO ARGUMENTS

Item at position k / first k / drop k

Nat and list move together: `(sub1 n)` and `(rest lst)`. Check `empty?` before using `first`.

```
(cond [(empty? lst) …]
      [(zero? n) … (first lst) …]
      [else (f (sub1 n) (rest lst))])
```

Combine two lists element by element

Lockstep: both `rest` every call. Decide what unequal lengths mean.

```
(cond [(or (empty? a) (empty? b)) …]
      [else (combine (first a) (first b) (f (rest a) (rest b)))])
```

Merge two sorted lists / walk two lists at different speeds

Compare the firsts and advance whichever list "wins" (or both on a tie).

Join lists / add to the end

`(append lst1 lst2)`; one element at the end: `(append lst (list x))`. Writing it yourself: recurse on the first list only.

ACCUMULATORS AND WRAPPERS

Reverse, or build the answer while walking

Accumulator holds the answer; base returns it; wrapper starts it at `empty` / `0`.

```
(define (f/acc lst acc)
  (cond [(empty? lst) acc]
        [else (f/acc (rest lst) (update acc (first lst)))]))
(define (f lst) (f/acc lst start))
```

Needs the position, the previous element, or a running total

Accumulator holds context; still build the output with `cons`; base returns `empty`.

```
;; Keep the elements at positions 0, 2, 4, ...   (a Bool flag flips each step)
;; every-other/acc: (listof Any) Bool -> (listof Any)
(define (every-other/acc lst keep?)
  (cond [(empty? lst) empty]
        [keep? (cons (first lst) (every-other/acc (rest lst) (not keep?)))]
        [else (every-other/acc (rest lst) (not keep?))]))
;; every-other: (listof Any) -> (listof Any)
(define (every-other lst) (every-other/acc lst true))
(check-expect (every-other (list 'a 'b 'c 'd 'e)) (list 'a 'c 'e))
(check-expect (every-other empty) empty)
```

TRAP

Recursing on `(rest (rest lst))` to skip elements breaks the Rules (only `(rest lst)` is allowed) and crashes on odd lengths. Use a flag or a position counter instead.

Wrap-around (compare the last with the first)

Pass the original first element along unchanged as an extra parameter (`perimeter/v2`).

Process a string

Wrapper: `string->list`; helper on `(listof Char)`; `list->string` at the end if the answer is a Str.

```
(define (f s) (list->string (f/list (string->list s))))
```

NESTED DATA

For each row / sublist ...

One function per level. The outer recursion walks the rows and calls a row helper on `(first table)`. A fold over rows, a map over rows, or a filter of rows.

Look up / update by key

Association list: key is `(first (first al))`, value is `(second (first al))`. Update = `(cons (list key new) (delete key al))`.

All combinations / subsets / "for each element, two choices"

Solve the rest recursively, then expand that answer with the first element using a helper.

;; across the course

Stepping rules

Everything you need to trace a program by hand, in one place.

The rules

RULE	REWRITE
0 · built-in	`(f v1 … vn) ⇒ v` once every argument is a value. Includes `first rest cons? empty? + sqr symbol=? string->list` …
1 · and	Leading `true`s then a `false` ⇒ `false`; all `true` ⇒ `true`
1 · or	Leading `false`s then a `true` ⇒ `true`; all `false` ⇒ `false`
1 · not	`(not true) ⇒ false`, `(not false) ⇒ true`
2 · user function	All arguments values ⇒ body with every parameter replaced
constant	name ⇒ its value (one step)
3 · cond	`[false e]` pair dropped; `[true e]` or `[else e]` ⇒ e

STEPS: CHOOSING THE NEXT REWRITE

1. Find the leftmost spot where a rule applies to an expression whose parts are already values.
2. Apply exactly one rule there. Copy everything else unchanged.
3. In a `cond`, only the first question is ever being evaluated. In `and` / `or`, evaluate left to right and stop as soon as the rule fires.

ALREADY VALUES (DON'T STEP)

Numbers, `true`, `false`, symbols, characters, strings, `empty`, and lists of values written with `cons` or `list`: `(cons 1 (cons 2 empty))` and `(list 1 2)` are finished.

Worked trace: a list function

```
(define (len lst)
  (cond [(empty? lst) 0]
        [else (add1 (len (rest lst)))]))

(len (list 7))
⇒ (cond [(empty? (list 7)) 0] [else (add1 (len (rest (list 7))))])
⇒ (cond [false 0] [else (add1 (len (rest (list 7))))])
⇒ (cond [else (add1 (len (rest (list 7))))])
⇒ (add1 (len (rest (list 7))))
⇒ (add1 (len empty))
⇒ (add1 (cond [(empty? empty) 0] [else (add1 (len (rest empty)))]))
⇒ (add1 (cond [true 0] [else (add1 (len (rest empty)))]))
⇒ (add1 0)
⇒ 1
```

10 expressions, matching the stepper's count of 10 for a one-element list (6n + 4). Notice the whole body is copied in at each call with `lst` replaced, and the pending `add1` waits outside.

Worked trace: helpers and short-circuiting

```
(define (big? n) (> n 10))
(define (check lst) (and (cons? lst) (big? (first lst))))

(check empty)
⇒ (and (cons? empty) (big? (first empty)))
⇒ (and false (big? (first empty)))      ; short-circuit: first is never called
⇒ false

(check (list 12))
⇒ (and (cons? (list 12)) (big? (first (list 12))))
⇒ (and true (big? (first (list 12))))
⇒ (and true (big? 12))
⇒ (and true (> 12 10))
⇒ (and true true)
⇒ true
```

Counting steps

- The stepper's counter includes the starting expression, so it is one more than the number of ⇒ arrows in your trace.
- Built-ins are one step each, including `length`, `append` and `reverse` (but count those as linear when classifying efficiency).
- Use the language "Beginning Student with List Abbreviations" or your counts won't match the course's.

;; across the course

Traps and common mistakes

The errors that cost marks most often, grouped by where they bite. Read this list the night before.

SYNTAX AND TYPES

Extra parentheses

`(5)`, `((f x))`, `(empty)` all try to call something that isn't a function.

Arity

`(+ 5)`, `(* 5)`, `(and true)` are errors (need two or more). `(- 5)` and `(max 5)` are fine.

The wrong equality

`=` only for numbers, `symbol=?` only for symbols, `char=?` for characters, `string=?` for strings. Each errors on the wrong type. `equal?` and `member?` are banned.

Non-Bool where a Bool is needed

Arguments to `and` / `or` / `not` and every `cond` question must be `true` or `false`.

No else, no match

A `cond` where every question is false and there's no `else` errors: "all question results were false".

Define before use, never redefine

Including using one name for a constant and a function. Don't name parameters after built-ins like `list`.

Subtraction with many arguments

`(- 10 1 2)` is 7 (10 – 1 – 2), not 11.

Division leaves Nat

`(/ 7 2)` is 7/2. Averages and ratios are `Num`, not `Nat`.

quotient/remainder on non-integers

Error on Rats like 3/2. Sign of `remainder` follows the first argument.

INEXACT NUMBERS AND TESTING

check-expect on an inexact result

Is an error, not just a failure. Any test involving `sqrt`, trig, `pi`, `log` → `check-within`.

= on inexact values

`(= (sin pi) 0)` is false. Compare with a tolerance.

Missing coverage

Every `cond` question needs a test where it's true and one where it's false. Test at each threshold and on both sides.

Bool conds

`(cond [A true] [else false])` is just `A`. Also flatten `[else (cond …)]`.

LISTS

first/rest on empty

Always test `empty?` first in the `cond`. `second` / `third` also need long enough lists.

Element vs list

`(first lst)` is an element; `(rest lst)` is a list. Comparing `(first lst)` to `(rest lst)` is a type error; you meant `(second lst)`.

cons is not append

`(cons (list 1) (list 2))` is `(list (list 1) 2)`. `(cons 1 2)` errors. To add at the end: `(append lst (list x))`.

Base case of the wrong type

A function producing a list returns `empty` at the base, not `0` or `false`. `insert` into empty returns `(list n)`, not `empty`.

Keep/drop swapped

Forgetting `(cons (first lst) …)` loses elements; consing in the drop branch removes nothing.

Recursing on lst instead of (rest lst)

Infinite recursion. Recursing on `(rest (rest lst))` breaks the Rules and skips neighbours.

(list empty) is not empty

It has length 1. `length` counts only top-level elements.

Missing Requires

Non-empty lists (`last` , `largest`) and sorted lists (`insert` , `merge`) need a Requires line.

RECURSION AND EFFICIENCY

Same recursive call twice

Especially one in a `cond` question and one in its answer. Exponential. Call once and pass the result to a helper or `max` / `min` .

Linear work in every call

`length` , `append` , `reverse` or a list-walking helper inside each recursive step makes the function quadratic.

Base-case order with two changing arguments

Check every base case, and put the ones that make `first` unsafe before the ones that use `first` (`index` : `empty?` before `zero?`).

Illegal recursion

Only `(sub1 n)` with `zero?` , and `(rest lst)` with `empty?` . Not `(- n 2)` , `(quotient n 10)` , `(add1 n)` as the shrinking argument.

Nat recursion fed a non-Nat

Negative or fractional input never reaches 0. That's what the `Nat` in the contract protects against.

Off by one

`count-down` includes 0; `countdown` stops at 1; `count-up n` stops at n−1; `(times 3 5)` has 4 rows and 6 columns. Test n = 0 and n = 1.

ACCUMULATORS

Backwards output

Consing onto an accumulator reverses order. Use plain recursion, or `reverse` once in the wrapper.

Wrong starting value

The wrapper's initial accumulator: `empty`, `0`, `1`, `true`, or the first element. A product starting at 0 is always 0.

reverse inside the recursion

A red flag for an approach that's too complicated and probably quadratic.

STRINGS AND DICTIONARIES

Character order

Uppercase before lowercase; digits before letters. `"Zebra"` < `"apple"`, `"10"` < `"9"`.

Forgetting list->string

If the contract says `-> Str`, convert the list of characters back.

Spaces are characters

`#\space` appears in `string->list` output and counts toward length.

Banned string functions

Only `string->list list->string string? string=? string<?` (and the other comparisons) plus the char comparisons. No `string-length`, `substring`, `char-upcase`.

Lookup returns the pair

The value is `(second (first al))`. Failure returns `empty`, so the contract is `(anyof Num empty)`.

add order

The new pair goes on the front, after deleting any old pair with that key. Expected values in tests must match that order.

;; across the course

Practice problems

Exam-style questions in roughly lecture order. Try each on paper before opening the solution. Every coded solution here has been run against its tests.

1 · Prefix and steps L01

Translate (5 + 3) · 2 − 12 ÷ 4 into Racket and show every substitution step.

SOLUTION

```
(- (* (+ 5 3) 2) (/ 12 4))
⇒ (- (* 8 2) (/ 12 4))
⇒ (- 16 (/ 12 4))
⇒ (- 16 3)
⇒ 13
```

```
(check-expect (- (* (+ 5 3) 2) (/ 12 4)) 13)
```

2 · De Morgan L01

Rewrite `(not (and (> x 0) (not (symbol? s))))` without any `not` applied to `and` / `or`, and with no `not` around a comparison.

SOLUTION

`(or (not (> x 0)) (not (not (symbol? s))))` → `(or (<= x 0) (symbol? s))`.

3 · Step a cond L03

```
(define (kind n)
  (cond [(< n 0) 'neg]
        [(zero? (remainder n 2)) 'even]
        [else 'odd]))
```

Show every step of `(kind 3)`. Then list a minimal set of tests that gives full coverage.

SOLUTION

```
(kind 3)
⇒ (cond [(< 3 0) 'neg] [(zero? (remainder 3 2)) 'even] [else 'odd])
⇒ (cond [false 'neg] [(zero? (remainder 3 2)) 'even] [else 'odd])
⇒ (cond [(zero? (remainder 3 2)) 'even] [else 'odd])
⇒ (cond [(zero? 1) 'even] [else 'odd])
⇒ (cond [false 'even] [else 'odd])
⇒ (cond [else 'odd])
⇒ 'odd
```

```
(define (kind n)
  (cond [(< n 0) 'neg]
        [(zero? (remainder n 2)) 'even]
        [else 'odd]))
(check-expect (kind -1) 'neg)    ; Q1 true
(check-expect (kind 0) 'even)    ; Q1 false (boundary), Q2 true
(check-expect (kind 3) 'odd)     ; Q2 false
```

---

4 · Sum of squares L04

Write `sum-squares: Nat -> Nat` producing $0^2 + 1^2 + \ldots + n^2$, following the design recipe.

SOLUTION

```
;; Sum the squares of 0..n.
;; sum-squares: Nat -> Nat
(define (sum-squares n)
  (cond [(zero? n) 0]
        [else (+ (sqr n) (sum-squares (sub1 n)))]))
(check-expect (sum-squares 0) 0)
(check-expect (sum-squares 1) 1)
(check-expect (sum-squares 3) 14)
```

---

5 · Count divisors (wrapper + fixed argument) L04

Write `num-divisors: Nat -> Nat` that counts how many of 1..n divide n exactly ($n \geq 1$). You can only recurse with `sub1`.

SOLUTION

Recurse on a second Nat k counting down from n; n itself never changes.

```
;; How many of 1..k divide n?
;; div-count: Nat Nat -> Nat
(define (div-count n k)
  (cond [(zero? k) 0]
        [(zero? (remainder n k)) (add1 (div-count n (sub1 k)))]
        [else (div-count n (sub1 k))]))
;; num-divisors: Nat -> Nat
;; Requires: n >= 1
(define (num-divisors n) (div-count n n))
(check-expect (num-divisors 12) 6)
(check-expect (num-divisors 7) 2)
(check-expect (num-divisors 1) 1)
```

---

6 · Composite data L05

A Student is `(list Str Num Num)`: name, midterm mark, final mark. Write the data definition, a constructor, accessors, and `course-mark: Student -> Num` worth 40% midterm and 60% final.

SOLUTION

```
;; A Student is a (list Str Num Num)   ; name, midterm, final
(define (mk-student name mid fin) (list name mid fin))
(define (student-name s) (first s))
(define (student-mid s) (second s))
(define (student-final s) (third s))
;; course-mark: Student -> Num
(define (course-mark s)
  (+ (* 0.4 (student-mid s)) (* 0.6 (student-final s))))
(check-expect (course-mark (mk-student "ada" 80 90)) 86)
(check-expect (course-mark (mk-student "bo" 0 0)) 0)
```

---

7 · Count in a range L06

Write `count-in-range: Num Num (listof Num) -> Nat` counting elements x with lo ≤ x ≤ hi.

SOLUTION

```
(define (count-in-range lo hi lon)
  (cond [(empty? lon) 0]
        [(and (<= lo (first lon)) (<= (first lon) hi))
         (add1 (count-in-range lo hi (rest lon)))]
        [else (count-in-range lo hi (rest lon))]))
(check-expect (count-in-range 1 5 empty) 0)
(check-expect (count-in-range 1 5 (list 0 1 3 5 6)) 3)
```

---

8 · All the same? L06 · L07

Write `all-same?: (listof Sym) -> Bool`, true when every symbol is the same (an empty list counts as true).

SOLUTION

"All equal" is the same as "every neighbour pair is equal", so use the adjacent-pair shape.

```
(define (all-same? los)
  (cond [(or (empty? los) (empty? (rest los))) true]
        [(symbol=? (first los) (second los)) (all-same? (rest los))]
        [else false]))
(check-expect (all-same? empty) true)
(check-expect (all-same? (list 'a 'a 'a)) true)
(check-expect (all-same? (list 'a 'a 'b)) false)
```

---

9 · Filter and conditional map L07

(a) `remove-negatives`: drop every negative number. (b) `double-evens`: double the even numbers, leave odd ones alone.

SOLUTION

```
;; remove-negatives: (listof Int) -> (listof Int)
(define (remove-negatives loi)
  (cond [(empty? loi) empty]
        [(< (first loi) 0) (remove-negatives (rest loi))]
        [else (cons (first loi) (remove-negatives (rest loi)))]))
(check-expect (remove-negatives (list 3 -1 0 -5 2)) (list 3 0 2))

;; double-evens: (listof Int) -> (listof Int)
(define (double-evens loi)
  (cond [(empty? loi) empty]
        [(even? (first loi)) (cons (* 2 (first loi)) (double-evens (rest loi)))]
        [else (cons (first loi) (double-evens (rest loi)))]))
(check-expect (double-evens (list 1 2 3 4)) (list 1 4 3 8))
```

---

10 · Squash repeats L07

Write `squash: (listof Num) -> (listof Num)` that collapses runs of equal neighbours: `(list 1 1 2 2 2 3 1)` → `(list 1 2 3 1)`.

SOLUTION

```
(define (squash lon)
  (cond [(or (empty? lon) (empty? (rest lon))) lon]
        [(= (first lon) (second lon)) (squash (rest lon))]   ; drop this copy
        [else (cons (first lon) (squash (rest lon)))]))
(check-expect (squash (list 1 1 2 2 2 3 1)) (list 1 2 3 1))
(check-expect (squash empty) empty)
(check-expect (squash (list 4)) (list 4))
```

---

11 · Insert into a descending list L07

Write `insert-desc` for lists sorted from largest to smallest.

SOLUTION

```
;; insert-desc: Num (listof Num) -> (listof Num)
;; Requires: lon sorted in descending order
(define (insert-desc n lon)
  (cond [(empty? lon) (list n)]
        [(> n (first lon)) (cons n lon)]
        [else (cons (first lon) (insert-desc n (rest lon)))]))
(check-expect (insert-desc 5 (list 9 7 3 1)) (list 9 7 5 3 1))
(check-expect (insert-desc 10 (list 9)) (list 10 9))
(check-expect (insert-desc 0 (list 9)) (list 9 0))
```

---

12 · Classify the efficiency L08

```
(define (a lst) (cond [(empty? lst) 0]
                      [else (+ (length lst) (a (rest lst)))]))
(define (b lst) (cond [(empty? lst) 0]
                      [(> (b (rest lst)) 10) 1]
                      [else (add1 (b (rest lst)))]))
(define (c lst) (cond [(empty? lst) 0]
                      [(odd? (first lst)) (add1 (c (rest lst)))]
                      [else (c (rest lst))]))
```

Also: a function takes 5, 12, 19, 26 steps on lists of length 0–3. What's its formula?

SOLUTION

- `a` : **quadratic**. One recursive call, but `length` is linear and runs at every level.
- `b` : **exponential**. `(b (rest lst))` runs in the question and, when that's false, again in the answer.
- `c` : **linear**. Two recursive calls written, but in different answers, so only one runs.
- Differences are all 7 → **7n + 5**, linear.

---

13 · Fix the blow-up L08

```
(define (smallest lon)
  (cond [(empty? (rest lon)) (first lon)]
        [(< (first lon) (smallest (rest lon))) (first lon)]
        [else (smallest (rest lon))]))
```

SOLUTION

```
;; smallest: (listof Num) -> Num
;; Requires: lon is non-empty
(define (smallest lon)
  (cond [(empty? (rest lon)) (first lon)]
        [else (min (first lon) (smallest (rest lon)))]))
(check-expect (smallest (list 5 3 8 1 9)) 1)
(check-expect (smallest (list 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8
7 6 5 4 3 2 1)) 1)
```

---

14 · Remove the item at position n L09

Write `remove-at: Nat (listof Any) -> (listof Any)` (positions from 0; out of range leaves the list unchanged).

SOLUTION

```
(define (remove-at n lst)
  (cond [(empty? lst) empty]
        [(zero? n) (rest lst)]
        [else (cons (first lst) (remove-at (sub1 n) (rest lst)))]))
(check-expect (remove-at 1 (list 'a 'b 'c)) (list 'a 'c))
(check-expect (remove-at 0 (list 'a)) empty)
(check-expect (remove-at 5 (list 'a 'b)) (list 'a 'b))
```

---

15 · Pairwise max (lockstep) L09

Produce the larger of each pair of corresponding elements, stopping at the end of the shorter list.

SOLUTION

```
;; pairwise-max: (listof Num) (listof Num) -> (listof Num)
(define (pairwise-max a b)
  (cond [(or (empty? a) (empty? b)) empty]
        [else (cons (max (first a) (first b)) (pairwise-max (rest a) (rest b)))]))
(check-expect (pairwise-max (list 1 9 3) (list 4 2 8 7)) (list 4 9 8))
(check-expect (pairwise-max empty (list 1)) empty)
```

---

16 · Prefix of a list L09

Write `prefix?: (listof Num) (listof Num) -> Bool` : is the first list the start of the second?

SOLUTION

```
(define (prefix? p lon)
  (cond [(empty? p) true]              ; used up the prefix: success
        [(empty? lon) false]           ; prefix is longer
        [else (and (= (first p) (first lon)) (prefix? (rest p) (rest lon)))]))
(check-expect (prefix? empty (list 1)) true)
(check-expect (prefix? (list 1 2) (list 1 2 3)) true)
(check-expect (prefix? (list 1 3) (list 1 2 3)) false)
(check-expect (prefix? (list 1 2) (list 1)) false)
```

---

17 · Position of the first match L10

Write `index-of: Sym (listof Sym) -> (anyof Nat false)`, the position of the first occurrence, or `false` if absent.

SOLUTION

```
;; index-of/acc: Sym (listof Sym) Nat -> (anyof Nat false)
(define (index-of/acc target los pos)
  (cond [(empty? los) false]
        [(symbol=? target (first los)) pos]
        [else (index-of/acc target (rest los) (add1 pos))]))
(define (index-of target los) (index-of/acc target los 0))
(check-expect (index-of 'c (list 'a 'b 'c 'c)) 2)
(check-expect (index-of 'z (list 'a)) false)
```

---

18 · Nested first/rest L10

```
(define data (list (list 'a (list 1 2)) (list 'b (list 3 4)) (list 'c empty)))
```

Evaluate: (a) `(first (rest (first (rest data))))` (b) `(rest (first (rest (rest data))))` (c) `(length (first data))`

SOLUTION

```
(define data (list (list 'a (list 1 2)) (list 'b (list 3 4)) (list 'c empty)))
(check-expect (first (rest (first (rest data)))) (list 3 4))
(check-expect (rest (first (rest (rest data)))) (list empty))
(check-expect (length (first data)) 2)
```

(b) catches people: `(first (rest (rest data)))` is `(list 'c empty)`, and its `rest` is `(list empty)`, a one-element list.

19 · Count in a table L11

Write `count-in-table: Num (listof (listof Num)) -> Nat`.

SOLUTION

```
;; count-in-row: Num (listof Num) -> Nat
(define (count-in-row x row)
  (cond [(empty? row) 0]
        [(= x (first row)) (add1 (count-in-row x (rest row)))]
        [else (count-in-row x (rest row))]))
;; count-in-table: Num (listof (listof Num)) -> Nat
(define (count-in-table x t)
  (cond [(empty? t) 0]
        [else (+ (count-in-row x (first t)) (count-in-table x (rest t)))]))
(check-expect (count-in-table 2 (list (list 1 2 2) empty (list 2 3))) 3)
(check-expect (count-in-table 9 empty) 0)
```

20 · Count vowels L12

Count the lowercase vowels (a, e, i, o, u) in a string.

SOLUTION

```
;; vowel?: Char -> Bool
(define (vowel? c)
  (or (char=? c #\a) (char=? c #\e) (char=? c #\i) (char=? c #\o) (char=? c #\u)))
;; count-vowels/list: (listof Char) -> Nat
(define (count-vowels/list loc)
  (cond [(empty? loc) 0]
        [(vowel? (first loc)) (add1 (count-vowels/list (rest loc)))]
        [else (count-vowels/list (rest loc))]))
;; count-vowels: Str -> Nat
(define (count-vowels s) (count-vowels/list (string->list s)))
(check-expect (count-vowels "programming is fun") 5)
(check-expect (count-vowels "") 0)
```

21 · Replace a character L12

Write `replace-char: Char Char Str -> Str`.

SOLUTION

```
(define (replace-char/list old new loc)
  (cond [(empty? loc) empty]
        [(char=? old (first loc)) (cons new (replace-char/list old new (rest loc)))]
        [else (cons (first loc) (replace-char/list old new (rest loc)))]))
(define (replace-char old new s)
  (list->string (replace-char/list old new (string->list s))))
(check-expect (replace-char #\space #\- "a b c") "a-b-c")
```

---

22 · Dictionary update and filter L12

Using `lookup`, `add` and `marks` from L12: (a) `bump: Str Num AL -> AL` adds delta to a key's value, or adds the key with value delta if it's missing. (b) `keys-above: Num AL -> (listof Str)` lists the keys whose value is greater than k.

SOLUTION

```
(define (bump key delta al)
  (cond [(empty? (lookup key al)) (add key delta al)]
        [else (add key (+ delta (lookup key al)) al)]))
(check-expect (bump "bjones" 10 marks)
              (list (list "bjones" 74) (list "asmith" 87) (list "cwang" 87)))
(check-expect (bump "new" 5 empty) (list (list "new" 5)))

(define (keys-above k al)
  (cond [(empty? al) empty]
        [(> (second (first al)) k) (cons (first (first al)) (keys-above k (rest al)))]
        [else (keys-above k (rest al))]))
(check-expect (keys-above 80 marks) (list "asmith" "cwang"))
```

`lookup` runs twice in `bump`, but on the same arguments in a non-recursive function, so it's only linear, not a blow-up.

---

23 · Find the bug L07 · L09

Each of these has one bug. Find it and say which input exposes it.

```
(define (index n lst)                    ; (a)
  (cond [(zero? n) (first lst)]
        [(empty? lst) empty]
        [else (index (sub1 n) (rest lst))]))

(define (ascending? lon)                 ; (b)
  (cond [(or (empty? lon) (empty? (rest lon))) true]
        [(> (first lon) (second lon)) false]
        [else (ascending? (rest (rest lon)))]))

(define (insert n lon)                   ; (c)
  (cond [(empty? lon) empty]
        [(< n (first lon)) (cons n lon)]
        [else (cons (first lon) (insert n (rest lon)))]))
```

SOLUTION

- (a) Base cases in the wrong order: `(index 0 empty)` calls `(first empty)`. Check `empty?` first.
- (b) Skips neighbours: `(list 1 5 2 6)` returns true because 5 and 2 are never compared. Recur on `(rest lon)`.
- (c) Inserting into empty loses n: `(insert 5 empty)` gives `empty`, and inserting a largest value drops it. Base should be `(list n)`.

;; across the course

Allowed constructs

The list grows each lecture. "Can I use ...?" is answered by "is it on the list?".
Everything below is allowed as of L12.

LECTURE	NEWLY ALLOWED
L01	`( ) + - * / = < > <= >= and define` (constants) `expt false max min not or quotient remainder sqr true` ; types Bool Int Nat Rat
L02	`; abs acos asin atan cos define` (functions) `e exp inexact? log number? boolean? pi sin sqrt tan` ; type Num
L03	`[ ] check-expect check-within cond else symbol? symbol=? zero?` ; anyof, Sym
L04	`sub1` ; recursion (Rules v1)
L05	`cons cons? empty empty? first list? rest` ; Any
L06	`add1 length` ; listof; Rules v2
L07	`even? odd? second third`
L08	`list`
L09	`append` ; final Rules for Recursion
L10	`reverse` ; accumulative recursion (unless disallowed)
L11	nothing new
L12	`char? char=? char<? string? string=? string<? string->list list->string` (plus the other comparison forms like `char<=?` and `string>?`); types Char, Str

NOT ALLOWED

`member?` , `equal?` and anything documented as using `equal?` ; `fourth` and beyond; any other string or character function (`string-length` , `substring` , `char-upcase` , ...) without explicit permission. Read the whole first page of an assignment or exam for extra restrictions.

Built from the CS135 Fall 2026 lecture slides L01–L12. Every runnable code block on this page was checked with Racket 9.3 in the Beginning Student language with list abbreviations.